

Design of control systems for
Aeronautical and Space Vehicles
AER4420

Name	Sec/BN#
<i>Sherif Essam Esmail</i>	1/25
<i>Abdelrahman Essam</i>	1/28
<i>Abdelrahman Muhamed</i>	1/30
<i>Muhamed Reffat</i>	2/13
<i>Youssef Ibrahim Ead</i>	2/42

Autopilot Project

03/03/2025

This page is intentionally left blank!

Contents

1	AUTOPILOT LITERATURE REVIEW	5
1.1	Research questions	5
1.1.1	Definition of Autopilot	5
1.1.2	Main Objective	5
1.1.3	Early Autopilot Development [1]	5
1.1.4	Inputs and Outputs of an Autopilot System	6
1.1.5	Human vs Autopilot	7
1.1.6	Stability Augmentation and Control Augmentation Systems [2]	7
1.1.7	Onboard Sensors	8
1.1.8	Nonmesurable States	8
1.1.9	Fly-By-Wire (FBW) [3]	10
1.2	Flight Mechanics Review	12
1.2.1	a) General Rigid Body Dynamics Equations in 3D (12 Equations)	12
1.2.2	b) Classification into Kinetics and Kinematics	
	Kinetic Equations (6 in total):	13
1.2.3	c) Additional Equations for Fixed-Wing Airplane Equations of Motion (EOM)	14
1.2.4	d) Assumptions in Deriving Airplane Equations of Motion.	14
1.2.5	e) Mathematical Classification of the Airplane EOM.	15
1.2.6	f) Difference Between Body Axes and Earth (Inertial) Axes.	15
1.2.7	g) Difference Between Pitch Angle (θ) vs. Angle of Attack (α) and Sideslip Angle (β) vs. Heading Angle (ψ).	15
1.2.8	h) Attitude Representations: Advantages and Disadvantages	16
1.3	Numerical solution of ODEs	16
1.3.1	Some of the numerical solving algorithms for ODEs	16
1.3.2	RK4	21
2	AIRPLANE SIMULATOR PART I (RK4)	23
2.1	Simulink Model	23
2.2	Comparing Results	25
2.2.1	Sampling Time Selection in Aircraft Simulation	25
2.3	Error Computation	32
3	Airframe Model	33
3.1	Simulink Model	33
3.2	Code	35

3.2.1	Extracting Lateral Stability Derivatives	36
3.2.2	Airframe Model	36
3.3	Results	36
A	MATLAB Codes	43
	Bibliography	56

Chapter 1

AUTOPILOT LITERATURE REVIEW

1.1 Research questions

1.1.1 Definition of Autopilot

An **autopilot** is a control system designed to operate a vehicle, such as an *aircraft, missile, or spacecraft*, with minimal human intervention by automatically regulating its trajectory and stability

1.1.2 Main Objective

The primary objectives of an autopilot system are as follows:

1. **Trajectory Maintenance:** Ensuring that the vehicle follows a predefined flight path by continuously adjusting control surfaces
2. **Stability Augmentation:** Enhancing vehicle stability by compensating for disturbances such as turbulence or atmospheric variations
3. **Workload Reduction:** Minimizing pilot intervention by automating routine control tasks, allowing operators to focus on higher-level decision-making
4. **Operational Efficiency:** Improving fuel consumption and accuracy in navigation, which contributes to safety and effectiveness

1.1.3 Early Autopilot Development [\[1\]](#)

1.1.3.1 Wright Brothers' Innovation (1903):

- Designed an unstable but controllable aircraft, improving maneuverability and reducing susceptibility to atmospheric gusts.
- Early pioneers (Lilienthal, Pilcher, Chanute, Langley) favored inherent stability, making piloting easier but reducing maneuverability.
- The Wright brothers' approach made flying more demanding, pushing the need for automatic pilots (autopilots).

1.1.3.2 Early Autopilot Development:

- "Spiral divergence": A common issue in aircraft where small disturbances cause increasing instability.
- Early autopilots aimed to correct deviations and restore the aircraft's flight attitude.
- **1912: First autopilot tested in a Curtiss flying boat, using gyroscopes and servo motors.**
- Sperry Aeroplane Stabilizer (1914): Developed by Sperry Gyroscope Company, led by Dr. E. A. Sperry.
- Won a 50,000-franc safety prize from the Aero Club of France, demonstrated by Lawrence Sperry.
- 1933: Wiley Post & Gyropilot: First pneumatic-hydraulic Gyropilot installed in his plane, Winnie May, Allowed autonomous flight, enabling Post to briefly sleep during an around-the-world flight in 8 days.
- 1947: Fully Automatic Flight: US Air Force C-54 equipped with Sperry A-12 autopilot & Bendix throttle control completed a transatlantic flight without manual control, Program stored on punched cards controlled the flight.
- Challenges in Modern Jets: Early autopilots focused on *maintaining attitude & heading*, Jet aircraft introduced new dynamic stability issues, like *short-period oscillations & Dutch roll*, Human pilots cannot correct rapid oscillations, requiring automatic damping systems, First successful automatic Dutch roll suppression appeared in YB-49 jet bomber.

1.1.4 Inputs and Outputs of an Autopilot System

1.1.4.1 Inputs (Desired/Commanded States)

The autopilot system relies on inputs that define the desired flight parameters. These inputs are typically set by the pilot or a flight management system (FMS). Common inputs include:

- **Altitude:** The target altitude to maintain (e.g., 35,000 feet).
- **Heading:** The desired compass direction (e.g., 270° for west).
- **Airspeed:** The target speed through the air (e.g., 250 knots).
- **Vertical Speed:** The desired rate of climb or descent (e.g., 1,000 feet per minute).
- **Pitch and Roll Angles:** The desired aircraft attitude for stability and maneuvering.
- **Navigation Data:** Commands from the FMS or GPS, such as waypoints or flight paths.

1.1.4.2 Outputs (Actuator Commands)

The autopilot generates commands to control the aircraft's actuators, which adjust the control surfaces or engines to achieve the desired states. The primary outputs include:

- **Ailerons:** Control roll by moving up or down to bank the aircraft left or right.
- **Elevators:** Control pitch by moving up or down to raise or lower the nose.
- **Rudder:** Control yaw by moving left or right to adjust the aircraft's heading.
- **Throttle/Thrust:** Adjust engine power to maintain or change airspeed.
- **Spoilers:** Assist in roll control or speed reduction (in some systems).
- **Flaps/Slats:** Adjust lift and drag during specific phases of flight (e.g., landing).

1.1.4.3 Control Loop Process

The autopilot operates as a closed-loop control system, continuously monitoring and adjusting the aircraft's state. The process involves:

1. **Sensing:** The autopilot receives real-time data from sensors (e.g., gyroscopes, accelerometers, GPS, altimeters, airspeed sensors).
2. **Comparison:** The autopilot compares the current state (e.g., actual altitude) with the desired state (e.g., target altitude).
3. **Error Calculation:** The difference (error) between the desired and actual state is calculated.
4. **Control Algorithm:** The autopilot uses control algorithms (e.g., PID controllers) to determine the necessary actuator commands to minimize the error.
5. **Actuation:** The autopilot sends commands to the actuators to adjust the control surfaces or engines, bringing the aircraft closer to the desired state.

1.1.4.4 Example

- **Input:** Desired altitude = 35,000 feet, current altitude = 34,500 feet.
- **Output:** Autopilot commands the elevators to pitch the nose up slightly and adjusts the throttle to increase thrust, causing the aircraft to climb to 35,000 feet.

1.1.5 Human vs Autopilot

While autopilot reduces pilot workload by automating routine tasks, pilots remain essential for:

- Flight Planning and Pre-Flight Checks
- Takeoff and Initial Climb
- Monitoring the Autopilot System
- Handling Emergencies and Malfunctions
- Adjusting Flight Parameters
- Approach and Landing
- Post-Landing and Taxiing

1.1.6 Stability Augmentation and Control Augmentation Systems [2]

Such control systems are known as stability augmentation systems (SASs). If the augmentation system is intended to control the mode and to provide the pilot with a particular type of response to the control inputs, it is known as a control augmentation system (CAS). An example of this is a normal acceleration CAS, in which the pilot's inputs are intended to control the acceleration generated along the negative z-axis. The slow modes (phugoid and spiral) are controllable by a pilot. But since it is undesirable for a pilot to have to pay continuous attention to controlling these modes, an automatic control system is needed to provide "pilot relief." An autopilot is an automatic control system that provides both pilot relief functions and special functions such as automatic landing. The common types of SAS, CAS, and autopilot functions can be listed as follows:

Table 1.1: Comparison of SAS, CAS, and Autopilot Functions

SAS	CAS	Autopilots
Roll damper	Roll rate control	Pitch-attitude hold
Pitch damper	Pitch rate control	Altitude hold
Yaw damper	Normal acceleration	Speed/Mach hold
-	Lateral/directional control	Automatic landing
-	-	Heading hold/VOR hold
-	-	Turn coordination
-	-	Roll angle hold

1.1.7 Onboard Sensors

Table 1.2: Sensors

Sensor	Quantities Measured	Sampling Rates	Sensor Example
GPS	Position (latitude, longitude, altitude), Velocity	1-10 Hz	u-blox NEO-M8N
Gyroscope	Angular velocity (roll, pitch, yaw rates)	100-1000 Hz	InvenSense MPU-6050
Accelerometer	Linear acceleration (x, y, z axes)	100-1000 Hz	STMicroelectronics LSM6DS3
Magnetometer	Magnetic field (heading or yaw)	10-200 Hz	Honeywell HMC5883L
Barometer	Altitude (pressure-based)	10-50 Hz	Bosch BMP280
IMU	Combined motion data	100-2000 Hz	Analog Devices ADIS16470
Airspeed Sensor	Airspeed (velocity relative to air)	100-1000 Hz	Honeywell AMS 5812

1.1.8 Nonmesurable States

the *nonmesurable states* can be estimated from the other states only if the system is *observable*

1.1.8.1 Observability

A system is **observable** if its state can be reconstructed from output measurements over time. This property ensures that unmeasured states can be inferred using a mathematical model.

1.1.8.2 Observability Condition for Linear Systems

Consider a linear time-invariant (LTI) system:

$$\dot{x} = Ax + Bu, \quad (1.1.1)$$

$$y = Cx + Du, \quad (1.1.2)$$

where x is the state vector, u is the input, and y is the measured output.

The system is **observable** if the *observability matrix* has full rank:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix} \quad (1.1.3)$$

If $\text{rank}(\mathcal{O}) = n$ (where n is the number of states), then all states can be estimated.

1.1.8.3 Observability for Nonlinear Systems

For nonlinear systems, observability is analyzed using the *Observability Gramian* or by applying Lie derivatives. The system dynamics are given by:

$$\dot{x} = f(x, u), \quad (1.1.4)$$

$$y = h(x, u). \quad (1.1.5)$$

The nonlinear observability rank condition requires computing the Lie derivatives of $h(x, u)$ along $f(x, u)$.

1.1.8.4 Observer Design

If a system is observable, an *observer* (state estimator) can be designed to reconstruct non-measured states.

1.1.8.5 Luenberger Observer (Linear Systems)

For an observable LTI system, a Luenberger observer estimates the state:

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}), \quad (1.1.6)$$

where $\hat{x}$ is the estimated state, and L is the observer gain. The gain L is designed such that the eigenvalues of $(A - LC)$ are stable (negative real parts).

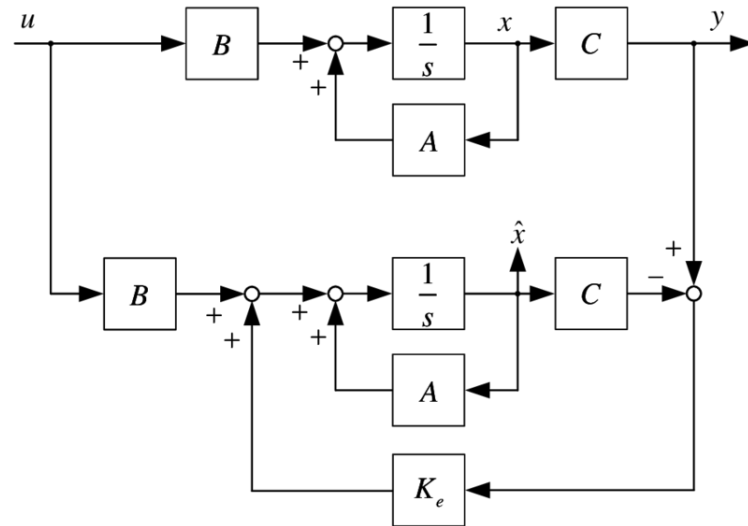


Figure 1.1.1: Observer Design Block Diagram

1.1.9 Fly-By-Wire (FBW) [3]

Fly-By-Wire (FBW) is a modern flight control system that replaces traditional mechanical linkages with electronic signal transmission. In FBW systems, pilot commands are interpreted by flight control computers, which then adjust the control surfaces accordingly.

1.1.9.1 Architecture of Fly-By-Wire Systems

An FBW system consists of the following key components:

1.1.9.2 Flight Control Computers (FCCs)

The FCCs process pilot commands, sensor inputs, and flight control laws to determine the required actuator movements. Modern FBW systems use redundant digital computers to enhance reliability.

1.1.9.3 Sensors and Feedback Systems

FBW systems incorporate gyroscopes, accelerometers, and other sensors to monitor aircraft motion and stability. These inputs are used to adjust control surface deflections dynamically.

1.1.9.4 Actuators

Electronic signals from the FCCs drive electro-mechanical or electro-hydrostatic actuators, which move the control surfaces such as ailerons, elevators, and rudders.

1.1.9.5 Advantages of Fly-By-Wire Systems

FBW systems offer several advantages over traditional mechanical and hydraulic control systems:

- **Weight Reduction:** Eliminates heavy mechanical linkages, reducing overall aircraft weight.
- **Improved Reliability:** Redundant digital computers enhance fault tolerance and system robustness.

- **Enhanced Maneuverability:** FBW allows for real-time adjustments based on sensor feedback, improving stability and responsiveness.
- **Maintenance Reduction:** Unlike hydraulic systems, FBW does not require extensive fluid checks, leak management, or pressure adjustments.
- **Safety Features:** Digital flight control laws can prevent dangerous maneuvers, such as stalls or excessive structural loads.

1.1.9.6 Autopilot Software VS Hardware [3]

Table 1.3: Comparison of Software and Hardware Autopilot Systems

Category	Software Autopilot		Hardware Autopilot	
Definition	Flight control algorithms and programs running on an autopilot system.		The physical computing unit executing autopilot software.	
Function	Manages UAV stabilization, navigation, control, and autonomous operations.		Provides processing power, sensor data acquisition, and control execution.	
Components	Source code (C/C++, Python), control algorithms, mission planning tools.		Microcontroller/processor, IMU (gyroscope, accelerometer, magnetometer), barometer, GPS, communication modules.	
Customization	Can be modified, updated, and adapted for different vehicles.		Limited by hardware specifications but can support different software.	
Examples	ArduPilot	(Runs on various hardware)	PX4 Pixhawk	(Hardware running PX4 software)
	PX4	(Open-source, used in different autopilot boards)	ArduPilot Mega (APM)	(Hardware for ArduPilot)
	MicroPilot Horizon	(Ground control software)	MicroPilot MP2x28xp	(Commercial UAV autopilot)
			DJI WooKong-M	(Proprietary autopilot hardware)
Availability	Open-source (ArduPilot, PX4) or proprietary (MicroPilot Horizon).		Open-hardware (PX4 Pixhawk) or proprietary (DJI WooKong, MicroPilot MP2x28xp).	
Development	Continuously updated with new features and improvements.		Hardware remains the same but can support firmware updates.	

1.2 Flight Mechanics Review

1.2.1 a) General Rigid Body Dynamics Equations in 3D (12 Equations)

A rigid body in 3D is fully described by 12 equations, split equally into 6 kinetic (dynamics) and 6 kinematic (geometric) equations.

Kinetic Equations (Dynamics)

These equations govern the forces and moments acting on the body.

1.2.1.1 Translational Dynamics (Newton's Second Law)

$$\dot{u} = (F_x/m) + r \cdot v - q \cdot w \quad (1.2.1)$$

$$\dot{v} = (F_y/m) + p \cdot w - r \cdot u \quad (1.2.2)$$

$$\dot{w} = (F_z/m) + q \cdot u - p \cdot v \quad (1.2.3)$$

Where:

- u, v, w = Velocity components along the body x -, y -, and z -axes
- F_x, F_y, F_z = External force components along the body axes
- m = Mass of the body
- p, q, r = Angular velocity components about the body x -, y -, and z -axes

1.2.1.2 Rotational Dynamics (Euler's Equations)

$$\dot{p} = (1/I_x) \cdot [M_x + (I_y - I_z) \cdot q \cdot r] \quad (1.2.4)$$

$$\dot{q} = (1/I_y) \cdot [M_y + (I_z - I_x) \cdot r \cdot p] \quad (1.2.5)$$

$$\dot{r} = (1/I_z) \cdot [M_z + (I_x - I_y) \cdot p \cdot q] \quad (1.2.6)$$

Where:

- I_x, I_y, I_z = Moments of inertia about the body axes
- M_x, M_y, M_z = External moment components about the body axes

Kinematic Equations (Geometry)

These equations relate the velocities and angular rates to the changes in position and orientation.

1.2.1.3 Translational Kinematics (Position Updates)

$$\dot{x} = u \cdot \cos \theta \cdot \cos \psi + v \cdot (\sin \varphi \cdot \sin \theta \cdot \cos \psi - \cos \varphi \cdot \sin \psi) + w \cdot (\cos \varphi \cdot \sin \theta \cdot \cos \psi + \sin \varphi \cdot \sin \psi) \quad (1.2.7)$$

$$\dot{y} = u \cdot \cos \theta \cdot \sin \psi + v \cdot (\sin \varphi \cdot \sin \theta \cdot \sin \psi + \cos \varphi \cdot \cos \psi) + w \cdot (\cos \varphi \cdot \sin \theta \cdot \sin \psi - \sin \varphi \cdot \cos \psi) \quad (1.2.8)$$

$$\dot{z} = -u \cdot \sin \theta + v \cdot \sin \varphi \cdot \cos \theta + w \cdot \cos \varphi \cdot \cos \theta \quad (1.2.9)$$

Where:

- x, y, z = Inertial (earth-fixed) coordinates
- φ, θ, ψ = Euler angles (roll, pitch, yaw)

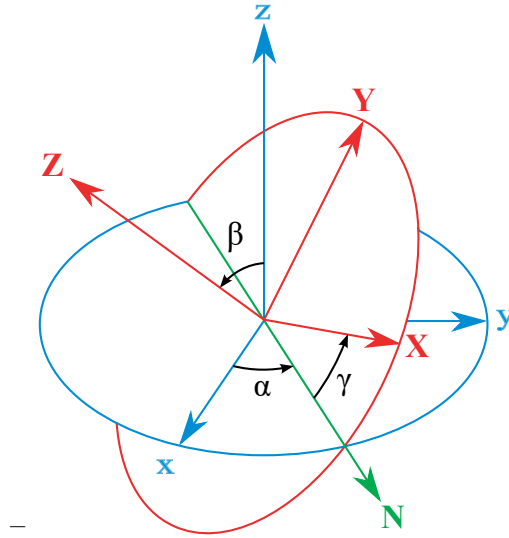


Figure 1.2.1: Euler Angles

1.2.1.4 Rotational Kinematics (Euler Angle Rates)

$$\dot{\varphi} = p + q \cdot \sin \varphi \cdot \tan \theta + r \cdot \cos \varphi \cdot \tan \theta \quad (1.2.10)$$

$$\dot{\theta} = q \cdot \cos \varphi - r \cdot \sin \varphi \quad (1.2.11)$$

$$\dot{\psi} = (q \cdot \sin \varphi + r \cdot \cos \varphi) / \cos \theta \quad (1.2.12)$$

These 12 equations together comprehensively describe the motion of a rigid body in 3D space.

1.2.2 b) Classification into Kinetics and Kinematics

Kinetic Equations (6 in total):

- Equations (1) - (3): Translational dynamics

- Equations (4) - (6): Rotational dynamics

Kinematic Equations (6 in total):

- Equations (7) - (9): Relate body velocities (expressed in body frame) to inertial position changes
- Equations (10) - (12): Relate body angular rates to Euler angle rates

1.2.3 c) Additional Equations for Fixed-Wing Airplane Equations of Motion (EOM)

To specialize the general RBD equations for a fixed-wing airplane, the following are added: •

1.2.3.1 Aerodynamic Force and Moment Models:

Aerodynamic forces are expressed as functions of dynamic pressure, wing area, and non-dimensional coefficients (e.g., C_x, C_y, C_z). Similarly, aerodynamic moments (roll L , pitch M , yaw N) are modeled using coefficients that depend on angle of attack, sideslip, and control surface deflections.

1.2.3.2 - Gravitational Force Projection:

The weight of the airplane is resolved along the body axes via the transformation from the inertial (or earthfixed) frame.

1.2.3.3 Velocity-to-Angle Relationships:

Equations that relate the components of velocity in the body frame to the airspeed, angle of attack (α), and sideslip angle (β).

- Control Input Effects:

Additional relations that quantify how control surface deflections affect the aerodynamic forces and moments.

1.2.4 d) Assumptions in Deriving Airplane Equations of Motion.

Common assumptions include:

- Rigid Body Approximation: The aircraft is assumed not to deform.
- Constant Mass and Inertia: Effects of fuel burn or payload shifts are neglected.
- Quasi-Steady Aerodynamics: Aerodynamic forces and moments are assumed to respond instantaneously to changes in flight conditions.
- Small Perturbations: Linearization is often performed around a trim condition.
- Neglect of High-Order Effects: Compressibility, viscous effects, and higher-order nonlinearities are typically ignored in preliminary analyses.
- Uniform Atmospheric Conditions: The air is assumed to have steady and uniform properties.

1.2.5 e) Mathematical Classification of the Airplane EOM.

Table 1.4: Classification of the Airplane EOM

Order	The equations are represented as first-order differential equations in state-space form.
Type	They are Ordinary Differential Equations (ODEs), with time being the independent variable.
Linearity	The full equations are nonlinear, although linearization is common near steady flight conditions.
Coupling	The equations are generally coupled, as translational and rotational dynamics interact. Under certain assumptions (such as decoupling of longitudinal and lateral-directional dynamics), the equations can be simplified to an uncoupled form.

1.2.6 f) Difference Between Body Axes and Earth (Inertial) Axes.

1.2.6.1 - Body Axes:

A coordinate system fixed to the airplane, typically defined as:

- x - axis: along the fuselage (forward),
- y - axis: towards the right wing,
- z -axis: downward.

The body axes rotate with the aircraft.

1.2.6.2 Earth (Inertial) Axes:

A fixed or quasi-fixed coordinate system relative to the Earth, often defined as North-East-Down (NED) or a similar scheme, serving as an absolute reference frame.

1.2.7 g) Difference Between Pitch Angle (θ) vs. Angle of Attack (α) and Sideslip Angle (β) vs. Heading Angle (ψ).

1.2.7.1 Pitch Angle (θ) vs. Angle of Attack (α):

The pitch angle θ is the orientation of the aircraft's longitudinal axis relative to the horizon, while the angle of attack α is the angle between the chord line (or x -axis) and the oncoming airflow. In steady, coordinated flight these angles are related, but during maneuvers or in the presence of wind, they can differ significantly.

1.2.7.2 Sideslip Angle (β) vs. Heading Angle (ψ):

The sideslip angle β is the angle between the aircraft's longitudinal axis and the relative wind, whereas the heading angle ψ is the navigational direction of the aircraft relative to a fixed reference (such as geographic North).

1.2.8 h) Attitude Representations: Advantages and Disadvantages

1.2.8.1 - Euler Angles:

- Advantages: Intuitive (roll, pitch, yaw) and straightforward.
- Disadvantages: Suffer from singularities (gimbal lock) when, for example, the pitch angle approaches ± 90 .

1.2.8.2 - Direction Cosine Matrix (DCM):

- Advantages: No singularity issues and provides a full transformation matrix.
- Disadvantages: Requires 9 elements along with orthogonality constraints, which can complicate numerical implementation.

1.2.8.3 - Quaternions:

- Advantages: Compact (4 parameters), computationally efficient, and free from singularities.
- Disadvantages: Less intuitive and involve a double-cover issue (i.e., q and $-q$ represent the same orientation).
- Axis-Angle Representation:
 - Advantages: Provides a clear geometric interpretation (rotation by a specific angle about a fixed axis).
 - Disadvantages: Less practical for sequential rotations or for numerical integration when rotations are small.

1.3 Numerical solution of ODEs

1.3.1 Some of the numerical solving algorithms for ODEs

Numerical solutions are approximate solutions to the differential equation where the analytical solution is hard to get or even not available as the case in airplane equations. The variation of the algorithms is due to the need for different approaches for solving the equations like: Accuracy requirements, stability considerations, computational efficiency, step size adaptability, and system constraints.

Reason for the Variation of Numerical Algorithms. The wide variety of numerical algorithms for solving Ordinary Differential Equations (ODEs) exists because different problems have unique characteristics that require specialized approaches. The main factors influencing algorithm selection include:

1. Accuracy Requirements - Some applications demand high precision (e.g., spacecraft navigation), while others prioritize speed (e.g., real-time control systems).
2. Stability Considerations - Certain ODEs, especially stiff systems (e.g., chemical reactions, flight dynamics at high speeds), require implicit methods for stable solutions.
3. Computational Efficiency - Simpler methods (e.g., Euler's) require fewer calculations but may need smaller time steps, while higher-order methods (e.g., Runge-Kutta) balance accuracy and efficiency.
4. Step Size Adaptability - Some methods (e.g., RK45) adjust the step size automatically to handle rapidly changing or smooth regions efficiently.

5. System Constraints - Real-time applications (e.g., autopilot, robotics) need algorithms that work under computational and time constraints.

1.3.1.1 Benefits of Algorithm Variation

Optimized Performance for Different Applications

- Euler's Method is useful for quick approximations.
- RK4 provides high accuracy for aerospace simulations.
- Implicit methods (e.g., Crank-Nicolson) are better for stiff problems like structural vibrations.

Balancing Accuracy & Speed

- High-order methods (RK4, RK45) are more precise but computationally expensive.
- Lower-order methods (Euler, Adams-Bashforth) are faster but may require smaller steps.

Handling Stiff vs. Non-Stiff Systems

- Stiff ODEs (e.g., high-speed aerodynamic models) require implicit methods for stability.
- Non-stiff ODEs (e.g., basic motion equations) can use explicit methods.

Adaptability to Problem Complexity

- Verlet integration conserves energy in orbital mechanics.
- Predictor-Corrector methods improve accuracy in long-duration flight simulations.

Real-Time Applications

- Some algorithms (e.g., fixed-step RK4) are used in autopilot systems to meet real-time constraints.
- Adaptive methods (RK45) are useful in simulations but may be too slow for real-time control

Table 1.5: Summary of Numerical Algorithms for Solving ODEs

Method	Order	Type	Pros	Cons
Euler's Method	1st	Explicit	Simple, fast	Low accuracy, unstable for large steps
RK4	4th	Explicit	High accuracy, widely used	Computationally expensive
RK45	Adaptive	Explicit	Efficient, variable step size	More complex
Adams-Bashforth	Variable	Explicit	Efficient, reuses previous values	Requires initialization
Adams-Moulton	Variable	Implicit	Stable, good for stiff problems	Computational overhead
Backward Euler	1st	Implicit	Stable for stiff ODEs	Requires solving nonlinear equations
Crank-Nicolson	2nd	Implicit	Stable, accurate	More complex than Backward Euler
Verlet Integration	2nd	Explicit	Energy-conserving	Requires two previous states

- Explicit Methods: Easier to implement but may require small step sizes for stability.
- Implicit Methods: More stable, especially for stiff problems, but require solving equations at each step.
- Adaptive Methods: Adjust step size automatically for better efficiency.
- Specialized Methods: Used for specific applications like orbital mechanics (Verlet) or flight control (Predictor-Corrector).

Choosing one algorithm for solving the Airplanes EOM, clearly state the (Initial conditions needed, Inputs needed in each iteration, and Outputs calculated in each iteration) The equation of motion of an airplane is derived from the rigid body motion equations. Those equations could be divided into five (or four) sets of equations:

Force equations

$$\begin{aligned}
X - mgS_\theta &= m(\dot{u} + qw - rv) \\
Y + mgC_\theta S_\Phi &= m(\dot{v} + ru - pw) \\
Z + mgC_\theta C_\Phi &= m(\dot{w} + pv - qu)
\end{aligned}$$

Moment equations

$$\begin{aligned}
L &= I_x \dot{p} - I_{xz} \dot{r} + qr(I_z - I_y) - I_{xz}pq \\
M &= I_y \dot{q} + rq(I_x - I_z) + I_{xz}(p^2 - r^2) \\
N &= -I_{xz} \dot{p} + I_z \dot{r} + pq(I_y - I_x) + I_{xz}qr
\end{aligned}$$

Angular velocity

$$\begin{aligned}
p &= \dot{\Phi} - \dot{\psi}S_\theta \\
q &= \dot{\theta}C_\Phi + \dot{\psi}C_\theta S_\Phi \\
r &= \dot{\psi}C_\theta C_\Phi - \dot{\theta}S_\Phi
\end{aligned}$$

Velocity equations

$$\begin{bmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \\ \frac{dz}{dt} \end{bmatrix} = \begin{bmatrix} C_\theta C_\psi & S_\Phi S_\theta C_\psi - C_\Phi S_\psi & C_\psi S_\theta C_\psi + S_\Phi S_\psi \\ C_\theta S_\psi & S_\Phi S_\theta S_\psi + C_\Phi C_\psi & C_\Phi S_\theta S_\psi - S_\Phi C_\psi \\ -S_\theta & S_\Phi C_\theta & C_\phi C_\theta \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix}$$

We need to get every derivative W.R.T. time so we will apply the small disturbance approximation where we could swap every variable with an initial known value and the change in this value. This approximation is only valid when the changes in those variables (states) are very small which could be seen almost at the cruise conditions throughout the journey of an aircraft.

The variables could be replaced as follows:

$$\begin{aligned}
u &= u_0 + \Delta u \\
v &= v_0 + \Delta v \\
w &= w_0 + \Delta w \\
p &= p_0 + \Delta p \\
q &= q_0 + \Delta q \\
r &= r_0 + \Delta r \\
X &= X_0 + \Delta X \\
Y &= Y_0 + \Delta Y \\
Z &= Z_0 + \Delta Z \\
M &= M_0 + \Delta M \\
N &= N_0 + \Delta N \\
L &= L_0 + \Delta L \\
\delta &= \delta_0 + \Delta \delta
\end{aligned}$$

Flight conditions are taken to be symmetric to simplify the equation to be:

$$\left. \begin{aligned}
\Delta X &= \frac{\partial X}{\partial u} \Delta u + \frac{\partial X}{\partial w} \Delta w \\
\Delta Y &= \frac{\partial Y}{\partial v} \Delta v + \frac{\partial Y}{\partial p} \Delta p + \frac{\partial Y}{\partial r} \Delta r \\
\Delta Z &= \frac{\partial Z}{\partial u} \Delta u + \frac{\partial Z}{\partial w} \Delta w + \frac{\partial Z}{\partial \dot{w}} \Delta \dot{w} + \frac{\partial Z}{\partial q} \Delta q
\end{aligned} \right\}$$

$$\Delta L = \frac{\partial L}{\partial v} \Delta v + \frac{\partial L}{\partial p} \Delta p + \frac{\partial L}{\partial r} \Delta r$$

$$\Delta M = \frac{\partial M}{\partial u} \Delta u + \frac{\partial M}{\partial w} \Delta w + \frac{\partial M}{\partial \dot{w}} \Delta \dot{w} + \frac{\partial M}{\partial q} \Delta q$$

$$\Delta N = \frac{\partial N}{\partial v} \Delta v + \frac{\partial N}{\partial p} \Delta p + \frac{\partial N}{\partial r} \Delta r$$

We will choose RK4 as our algorithm for solving the EOM, and here are the ICs, BCs, and the outputs of each iteration:

Initial conditions - Before starting the numerical integration, we need the initial state of the aircraft, which typically includes:

- Position: (initial location in a given coordinate system)
- Velocity: (initial velocity components in body frame)
- Attitude (Orientation): (initial roll, pitch, and yaw angles)
- Angular Velocity: (initial roll, pitch, and yaw rates)
- Mass & Inertia Properties: (aircraft mass and moments of inertia)

Input needed in each iteration - At each time step, the numerical algorithm needs updated information to compute the next state:

- Aerodynamic Forces & Moments:
 1. Lift, Drag, and Side Forces
 2. Moments about body axes
- Control Inputs (from Pilot or Autopilot):
 1. Throttle Setting
 2. Elevator Deflection
 3. Aileron Deflection
 4. Rudder Deflection
- Environmental Conditions:
 1. Wind Velocity Components
 2. Air Density
 3. Gravity

Outputs Calculated in Each Iteration -The numerical algorithm updates and computes the next state of the aircraft:

- Updated position
- Updated velocity components
- Updated attitude (orientation)
- Updated angular velocity
- New forces & moments
- kkkkkk5555

1.3.2 RK4

Solving [problem1](#) by RK4.

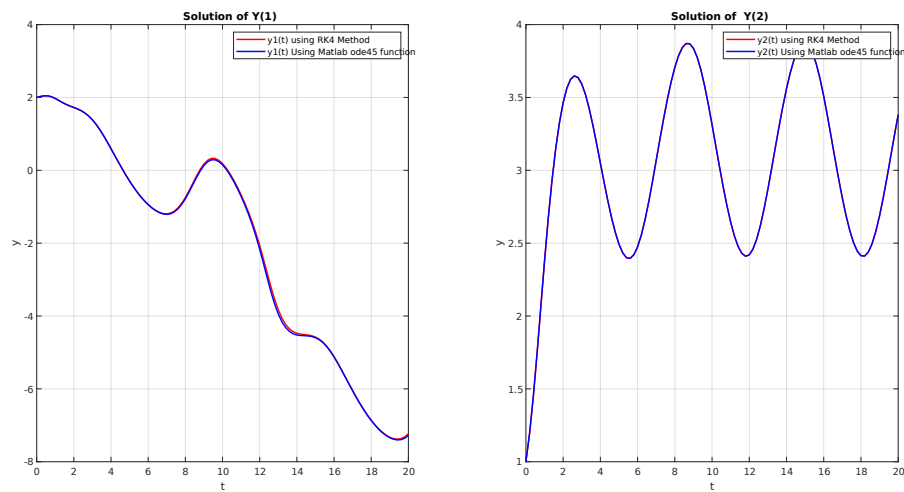


Figure 1.3.1: Our Code VS MATLAB

This page is intentionally left blank!

Chapter 2

AIRPLANE SIMULATOR PART I (RK4)

2.1 Simulink Model

To Verify and Validate the results obtained from the code that solved RBD equations using "Runge-Kutta" 4th order method. The code results was compared with the results from Simulink model that use the 6 DOF Euler angles block to simulate the RGB equations. The mass of the body was assumed to be constant and the properties defined for the 6 DOF Euler angles block is:

- Initial position in inertial axes (X_0, Y_0, Z_0)
- Initial Velocity in body axes (u_0, v_0, w_0)
- Inertial Euler Orientation $(\phi_0, \theta_0, \psi_0)$
- Initial body rotation rates (p_0, q_0, r_0)
- mass
- Inertia (3X3 Matrix)

These values were defined in the code then imported from the workspace.

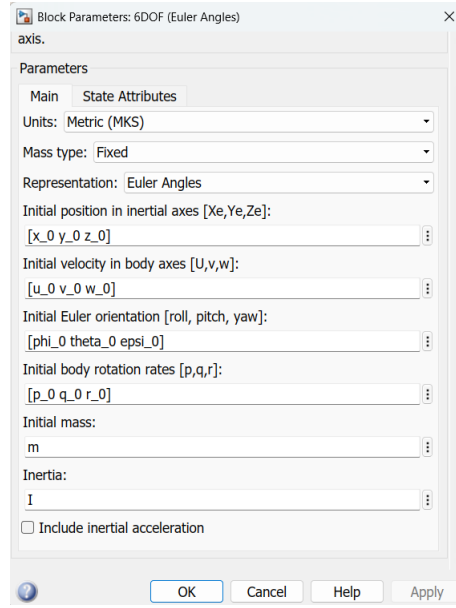


Figure 2.1.1: Input to the 6 DOF Euler angles block

$$\begin{aligned}
 mass &= 11 \text{ kg} \\
 I &= \begin{bmatrix} 1 & -2 & -1 \\ -2 & 5 & -4 \\ -1 & -4 & 0.2 \end{bmatrix} \text{ kg} \cdot \text{m}^2 \\
 [u, v, w, p, q, r, \phi, \theta, \psi, x, y, z]_{t=0} &= \\
 [10, 2, 0, 2 * \frac{\pi}{180}, 1 * \frac{\pi}{180}, 0 * \frac{\pi}{180}, 20 * \frac{\pi}{180}, 15 * \frac{\pi}{180}, 30 * \frac{\pi}{180}, 2, 4, 7]
 \end{aligned}$$

Figure 2.1.2: The Values Used for Euler block properties

$$\begin{aligned}
 m &= 11 \text{ kg} \\
 I &= \begin{bmatrix} 1 & -2 & -1 \\ -2 & 5 & -4 \\ -1 & -4 & 0.2 \end{bmatrix} \text{ kg} \cdot \text{m}^2 \\
 [u, v, w, p, q, r, \phi, \theta, \psi, x, y, z]_{t=0} &= \\
 [10, 2, 0, 2 * \frac{\pi}{180}, 1 * \frac{\pi}{180}, 0 * \frac{\pi}{180}, 20 * \frac{\pi}{180}, 15 * \frac{\pi}{180}, 30 * \frac{\pi}{180}, 2, 4, 7]
 \end{aligned}$$

The input to the 6 DOF Euler angles block is Forces and Moments which has values of:

$$\text{Forces} = [2 \quad 8 \quad 3] N$$

$$\text{Moments} = [14 \quad 20 \quad 7] N \cdot m$$

The simulation time was 25 s

Then the output values of the model was exported back to the code to be compared with the output values of the "Runge-Kutta" 4th order method code.

The output values exported back were:

- position in inertial axes (X, Y, Z)
- Initial Velocity in body axes (u, v, w)
- Inertial Euler Orientation (ϕ, θ, ψ)
- Initial body rotation rates (p, q, r)

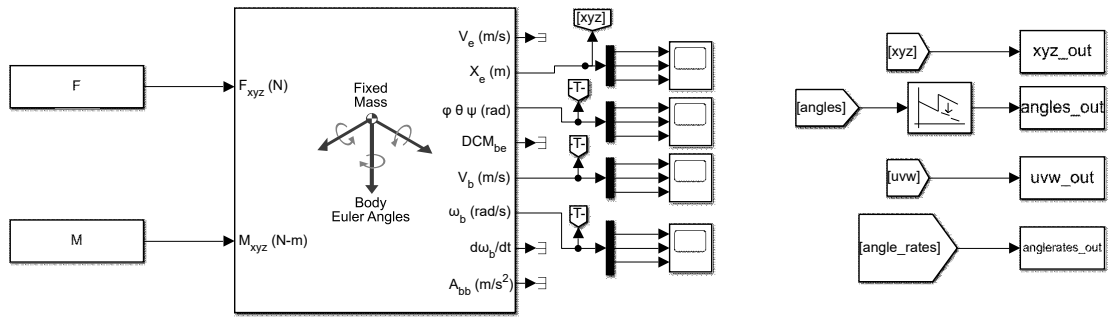


Figure 2.1.3: Simulink Model Used

2.2 Comparing Results

2.2.1 Sampling Time Selection in Aircraft Simulation

To determine the best number of samples for a 25-second simulation, we need to choose an appropriate sampling time T_s . The sampling time is critical in digital control and numerical integration when solving aircraft equations of motion using a numerical solver like Runge-Kutta 4th order (RK4).

2.2.1.1 Selecting an Appropriate Sampling Time

The choice of T_s depends on the fastest dynamics of the system. Based on digital control principles [4, 5], common sampling times for aircraft systems are:

- **Slow response systems (e.g., navigation, autopilot):** $T_s = 50$ ms (0.05 s)
- **Flight control (inner loops, stability augmentation):** $T_s = 10$ ms (0.01 s)
- **High-bandwidth actuator control:** $T_s = 1$ ms (0.001 s)

2.2.1.2 Calculating the Number of Samples

The total number of samples required for a 25-second simulation is:

$$N = \frac{T_{\text{simulation}}}{T_s} \quad (2.2.1)$$

For different sampling times:

- $T_s = 50 \text{ ms} \Rightarrow N = \frac{25}{0.05} = 500 \text{ samples}$
- $T_s = 10 \text{ ms} \Rightarrow N = \frac{25}{0.01} = 2500 \text{ samples}$
- $T_s = 1 \text{ ms} \Rightarrow N = \frac{25}{0.001} = 25000 \text{ samples}$

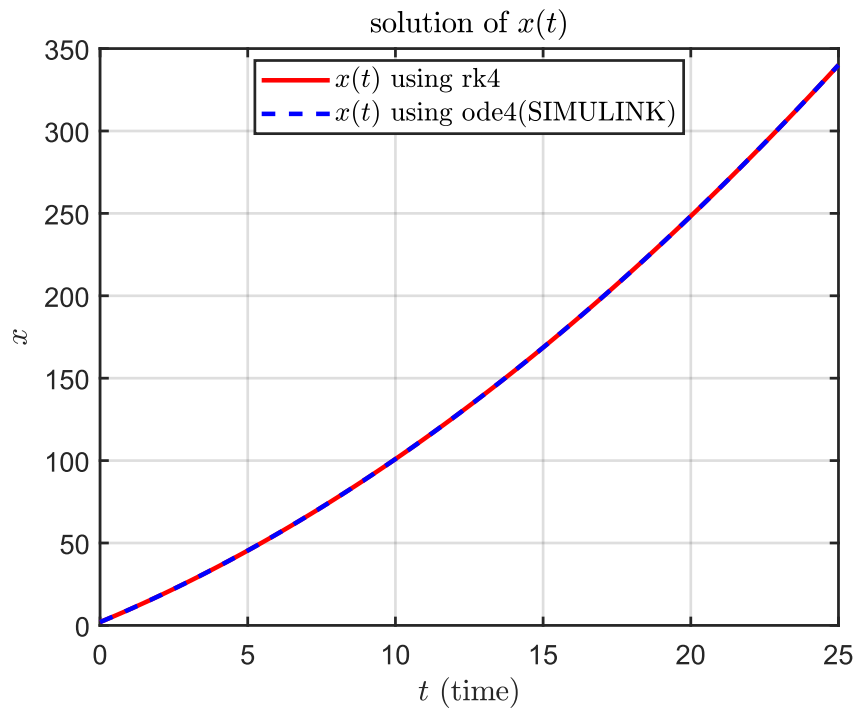
2.2.1.3 Numerical ODE Solver Considerations

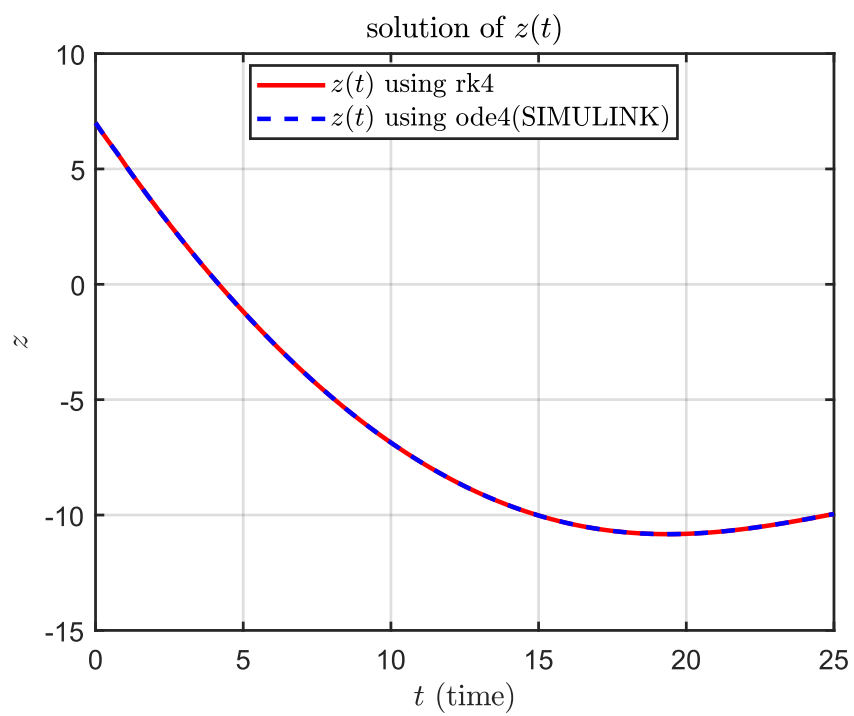
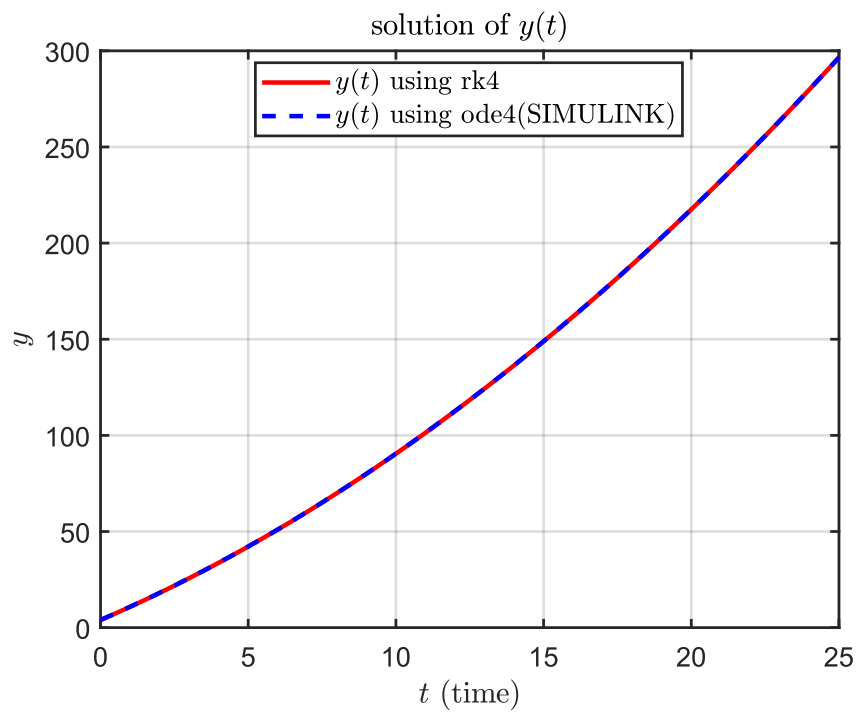
When using RK4 for solving the equations of motion, the internal solver step Δt should be smaller than T_s . As recommended in numerical integration literature [6, 7], a good practice is:

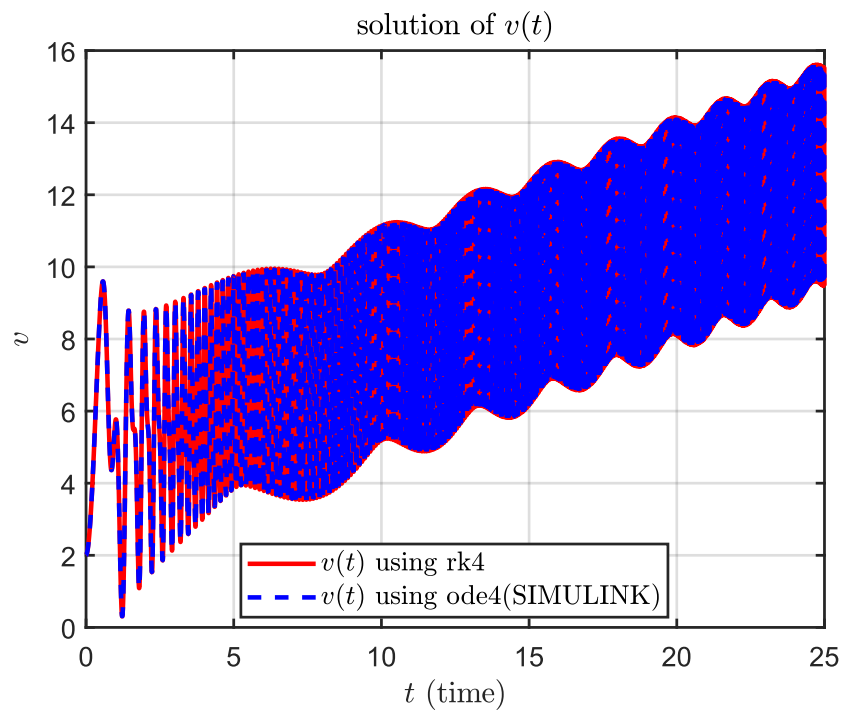
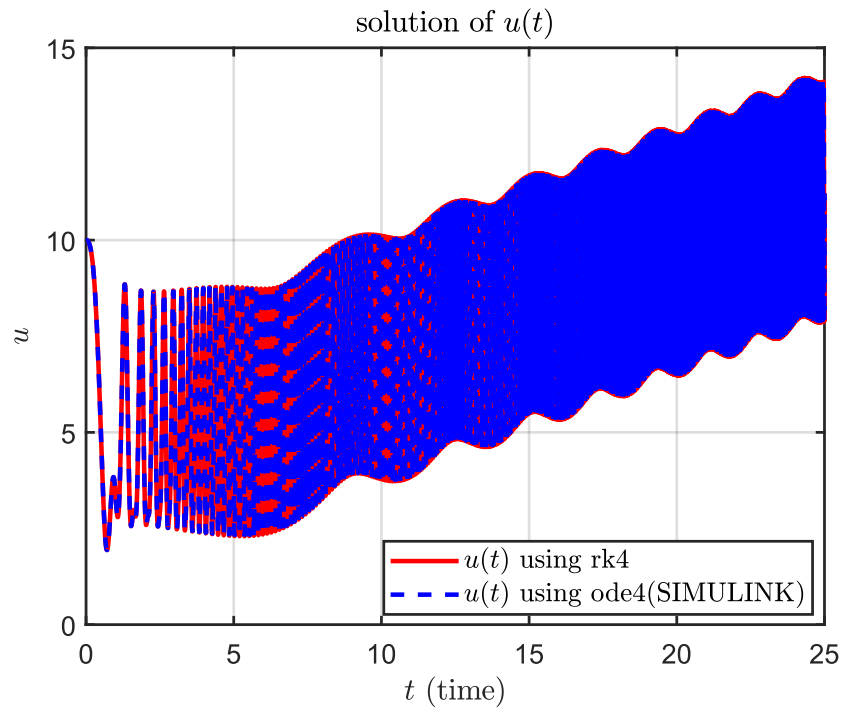
$$\Delta t = \frac{T_s}{5} \text{ or } \frac{T_s}{10} \quad (2.2.2)$$

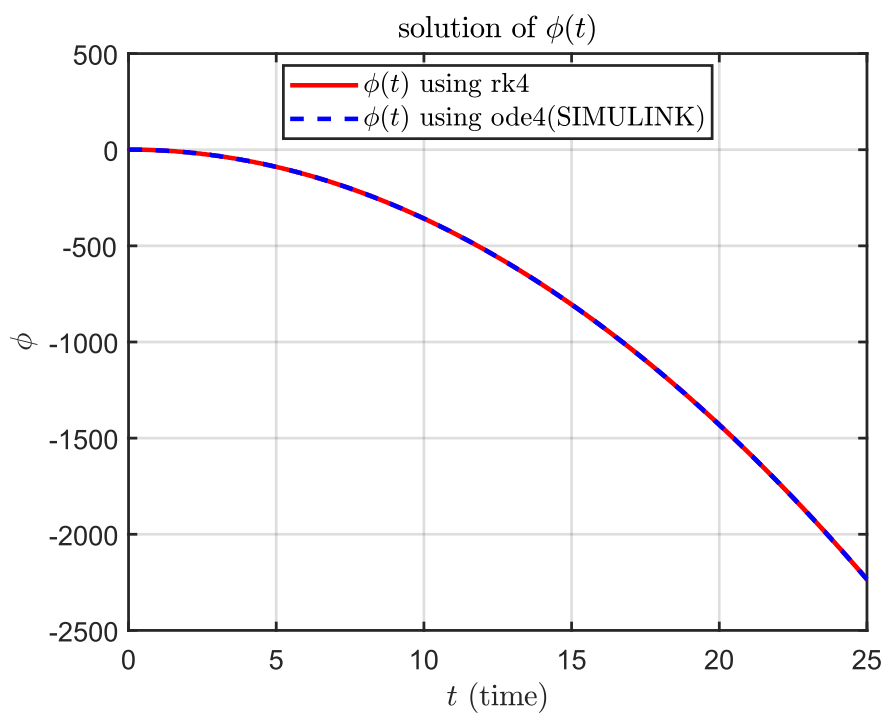
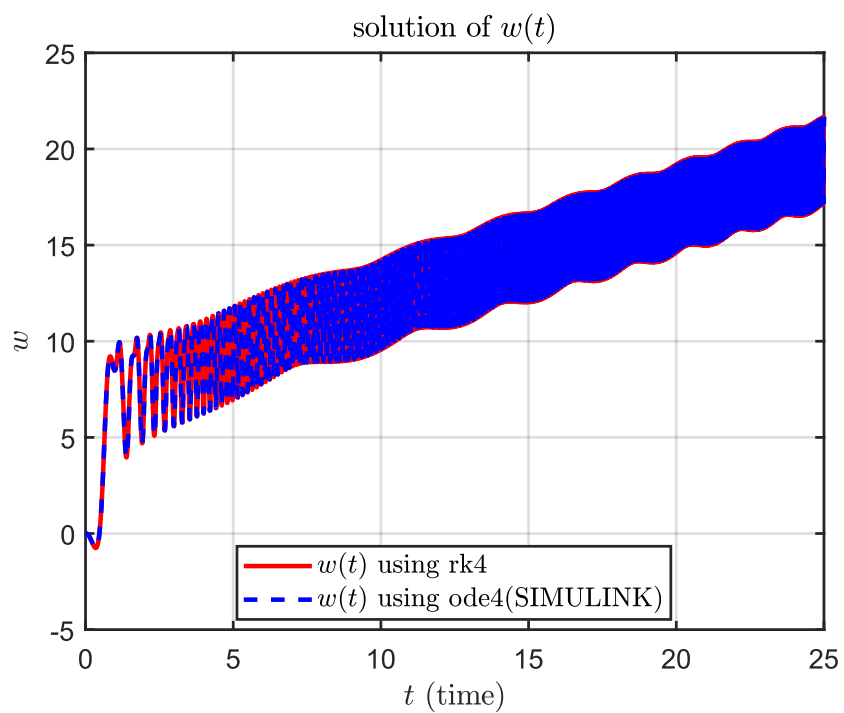
to ensure stability and accuracy.

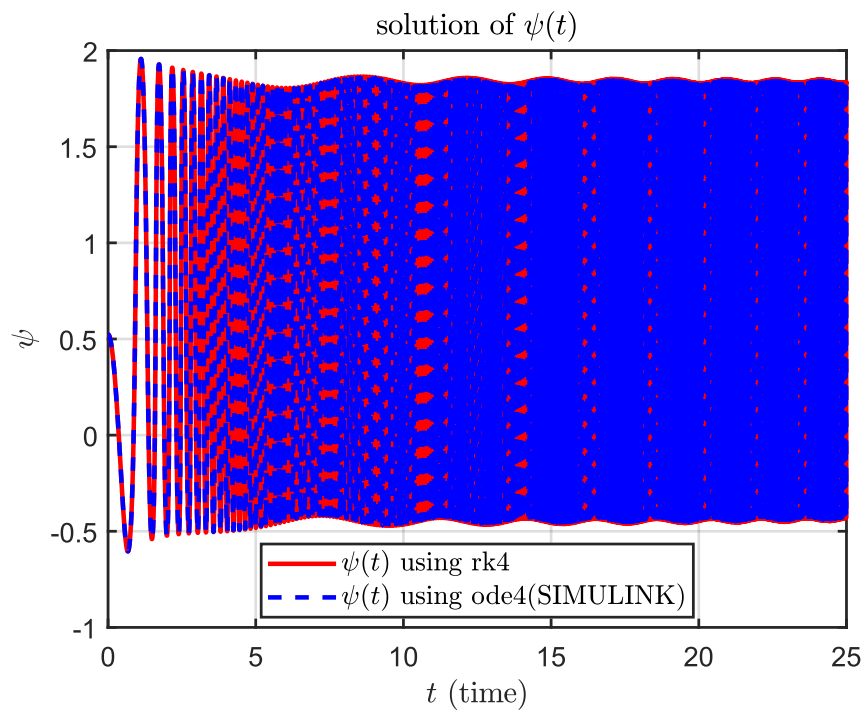
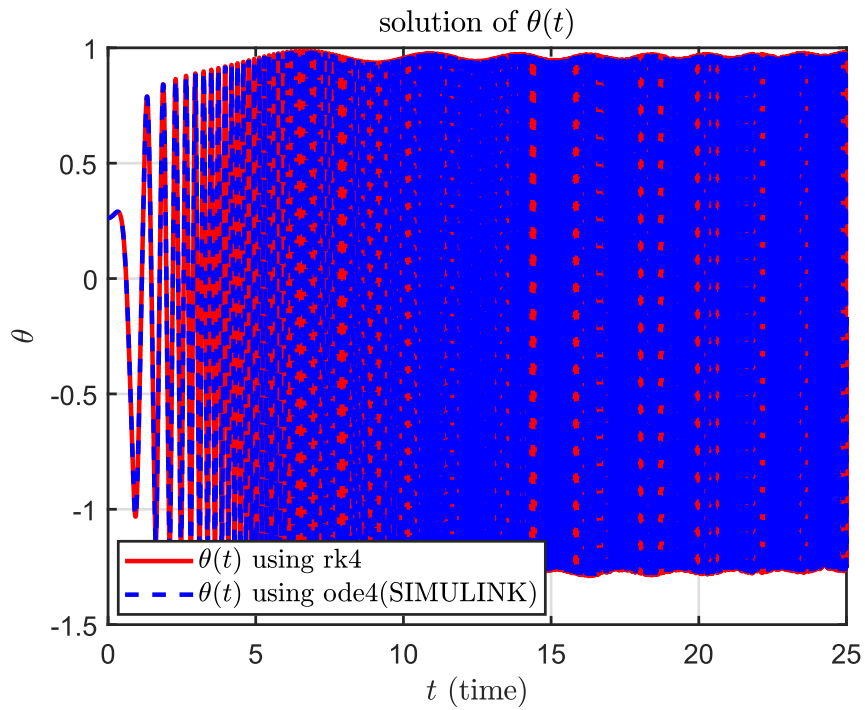
The figures comparing the values coming out of Simulink model and [AIRPLANE SIMULATOR PART I](#) code output are as follows:

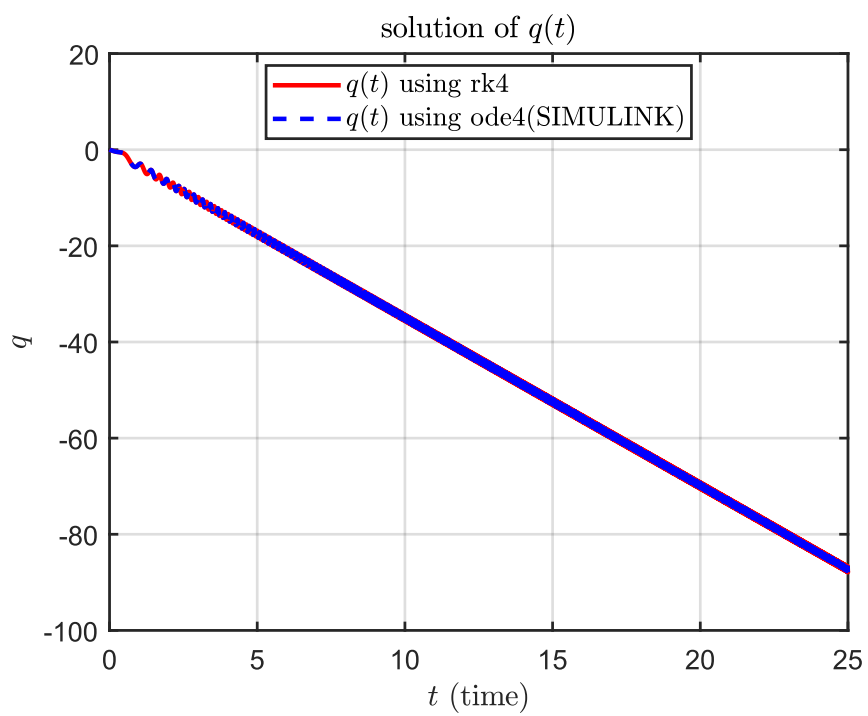
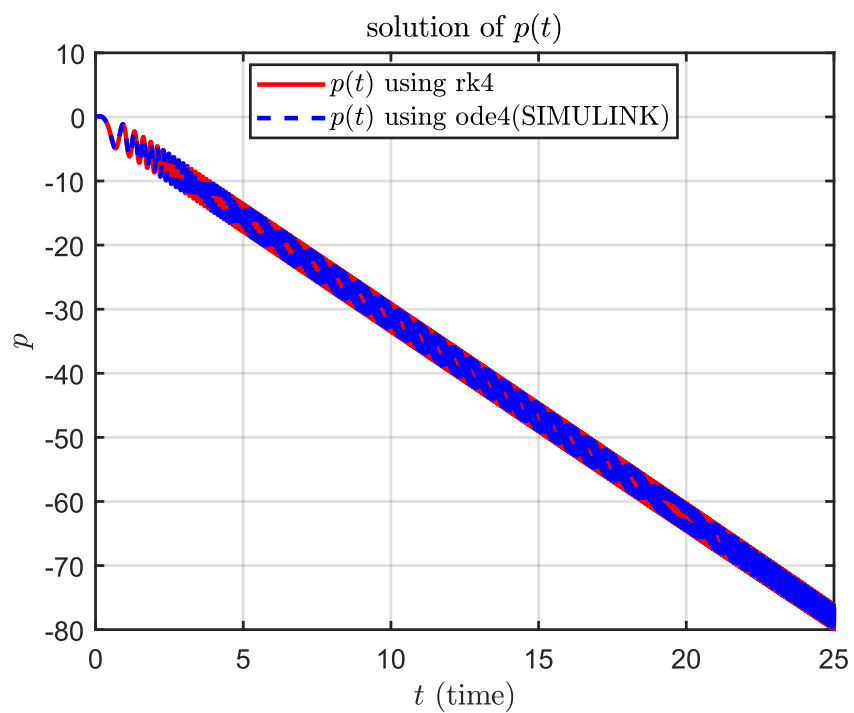


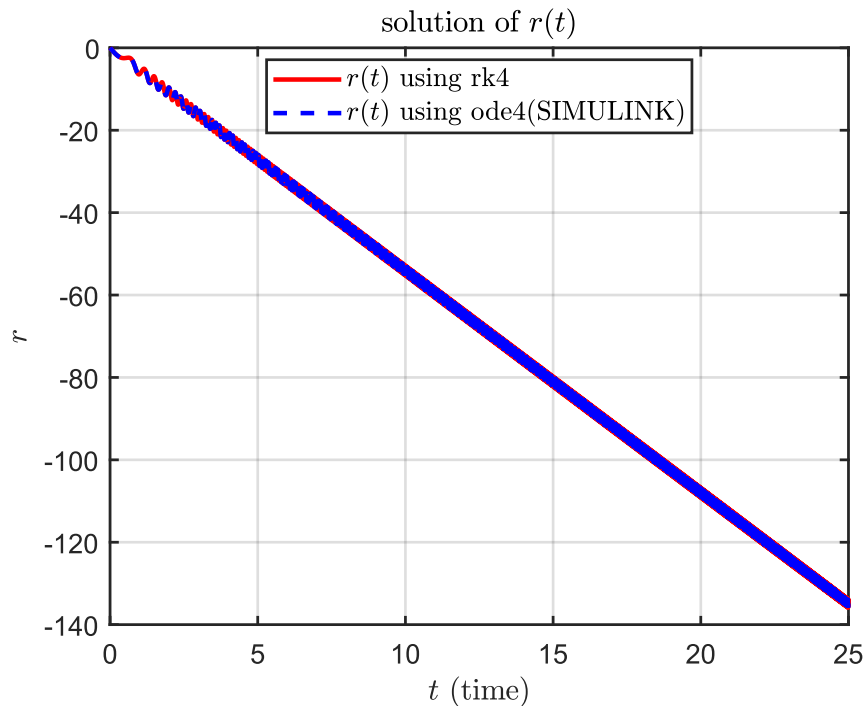












From the above figures it is clear that the two lines for both the code output and simulink simulation output coincide with each other demonstrating that the code results for solving RBD equations using "Runge-Kutta" 4th order method are valid.

2.3 Error Computation

This function `compareErrors` calculates the deviation between RK4 and Simulink solutions:

- Mean absolute error for each state variable.
- Root Mean Square Error (RMSE) to measure average deviation.
- Maximum error to identify the worst-case difference.

Mean Absolute Error (MAE) for each state:

```
1.0e-09 * [0.0037 0.0036 0.0027 0.0050 0.0016 0.0028 0.0031 0.0012 0.0027
0.1684 0.1931 0.0672]
```

Root Mean Square Error (RMSE) for each state:

```
1.0e-09 * [0.0053 0.0052 0.0039 0.0071 0.0022 0.0040 0.0049 0.0019 0.0041
0.2815 0.3240 0.1358]
```

Max Error for each state:

```
1.0e-09 * [0.0132 0.0133 0.0099 0.0156 0.0049 0.0089 0.0193 0.0053 0.0136
0.8145 0.9377 0.4579]
```


Chapter 3

Airframe Model

3.1 Simulink Model

This Simulink Model aims to calculate the (Aerodynamic & Thrust) (Forces & Moments) acting on an Airplane due to pilot input signals ($\delta_{aileron}$, δ_{rudder} , $\delta_{elevator}$, δ_{thrust}) knowing its stability & control derivatives at nominal flight condition to validate the results of RK4 code.

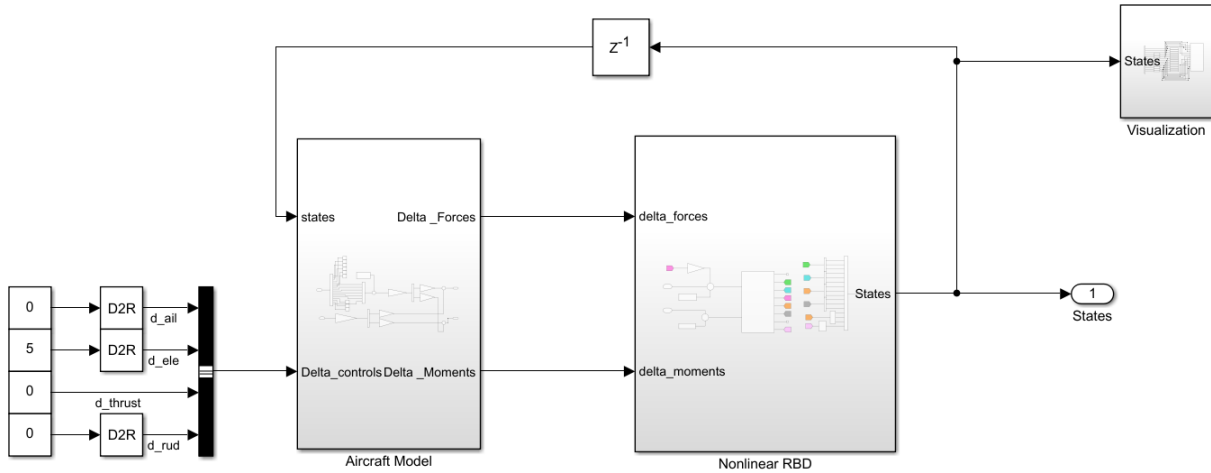


Figure 3.1.1: Full Simulink Model

The model consists of two main blocks namely the Aircraft Model and Nonlinear RBD. The Nonlinear RBD block calculates the 16 States necessary for the Aircraft Model to function which calculates the delta forces and moments and feeds them back to the Nonlinear Rigid Body Dynamics block creating a loop that simulates the aircraft. The whole Simulink model takes pilot input signals ($\delta_{aileron}$, δ_{rudder} , $\delta_{elevator}$, δ_{thrust}) as input and gives out many variables as output to determine the aircraft performance.

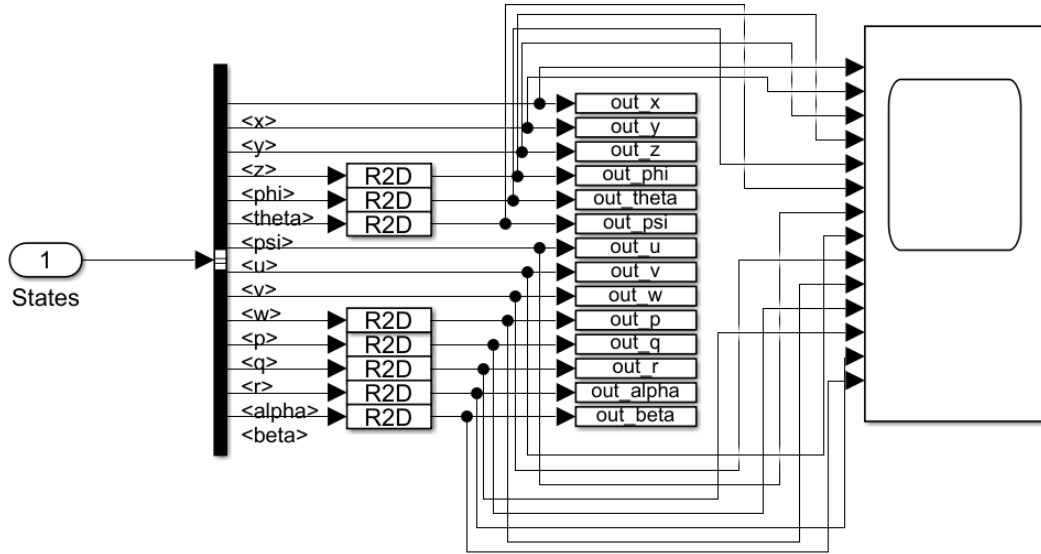


Figure 3.1.2: Visualization Block

The Model gives out Position (x, y, z) , Euler angles (ϕ, θ, ψ) , Velocities (u, v, w) , Angular Velocities (p, q, r) , and both angle of attack (α) and Side-slip angle (β) .

The Rigid Body Dynamics block uses Euler 6 DOF Block to calculate the 16 States. It takes forces and moments as input, The input forces are a result of Delta forces (ΔF) coming from the Aircraft model plus the gravity contribution in the body axis plus the gravity contribution after Euler angles change. Then the states are sent to the Aircraft Model.

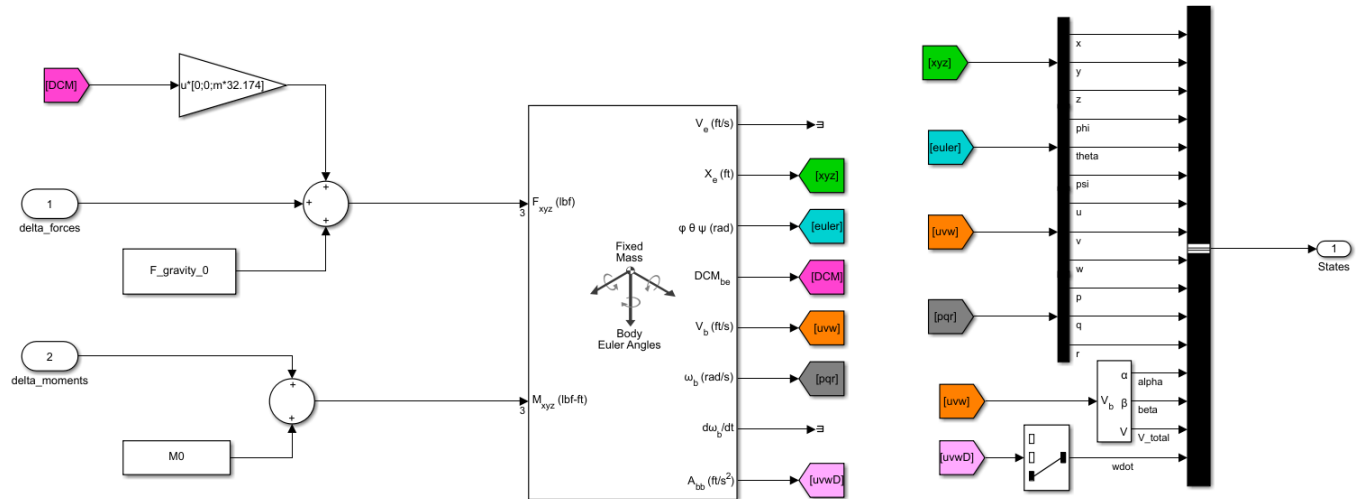


Figure 3.1.3: RBD Block

A unit delay is inserted between The two blocks as to not create an algebraic loop where the input is dependant on the output and it takes the initial conditions of the states as input

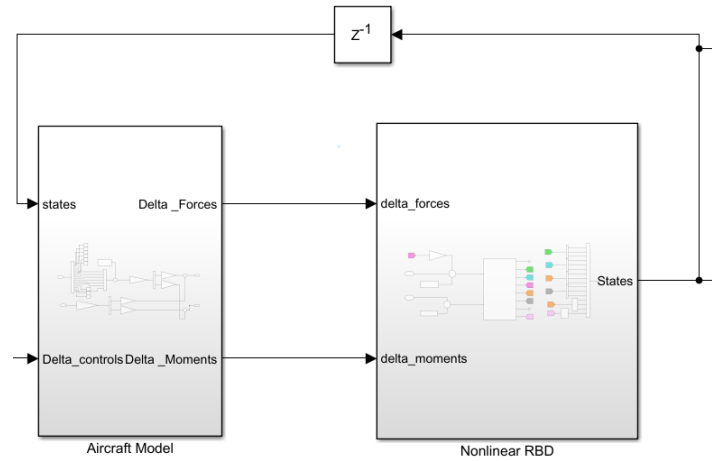


Figure 3.1.4: Unit Delay

The Aircraft Model uses the states from RBD block multiplies them by the states matrix and multiplies the control inputs of the pilot by the controls matrix and multiplies by the mass and moments of inertia to get the delta forces and moments.

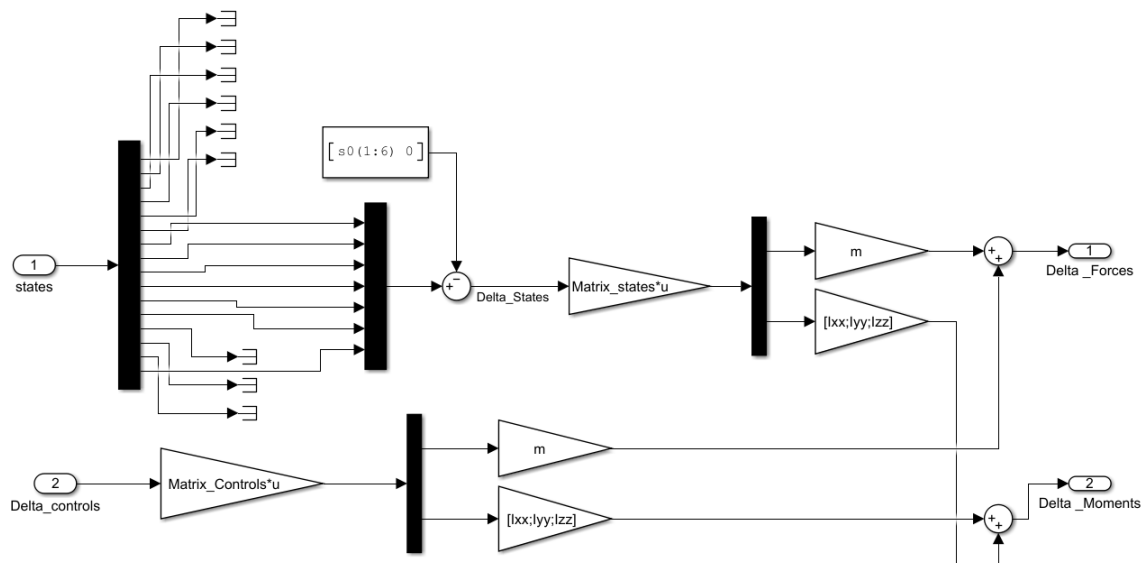


Figure 3.1.5: Aircraft Model Block

3.2 Code

the last chapter we made [A.3](#) that take the constant Forces and Moments and gives us the states. In that chapter we will calculate the Forces and moments in each time step using the Linearized model :

$$\begin{aligned}
\Delta X &= \frac{\partial X}{\partial u} \Delta u + \frac{\partial X}{\partial w} \Delta w + \frac{\partial X}{\partial \delta_e} \Delta \delta_e + \frac{\partial X}{\partial \delta_T} \Delta \delta_T \\
\Delta Y &= \frac{\partial Y}{\partial v} \Delta v + \frac{\partial Y}{\partial p} \Delta p + \frac{\partial Y}{\partial r} \Delta r + \frac{\partial Y}{\partial \delta_r} \Delta \delta_r \\
\Delta Z &= \frac{\partial Z}{\partial u} \Delta u + \frac{\partial Z}{\partial w} \Delta w + \frac{\partial Z}{\partial \dot{w}} \Delta \dot{w} + \frac{\partial Z}{\partial q} \Delta q + \frac{\partial Z}{\partial \delta_e} \Delta \delta_e + \frac{\partial Z}{\partial \delta_T} \Delta \delta_T \\
\Delta L &= \frac{\partial L}{\partial v} \Delta v + \frac{\partial L}{\partial p} \Delta p + \frac{\partial L}{\partial r} \Delta r + \frac{\partial L}{\partial \delta_r} \Delta \delta_r + \frac{\partial L}{\partial \delta_a} \Delta \delta_a \\
\Delta M &= \frac{\partial M}{\partial u} \Delta u + \frac{\partial M}{\partial w} \Delta w + \frac{\partial M}{\partial \dot{w}} \Delta \dot{w} + \frac{\partial M}{\partial q} \Delta q + \frac{\partial M}{\partial \delta_e} \Delta \delta_e + \frac{\partial M}{\partial \delta_T} \Delta \delta_T \\
\Delta N &= \frac{\partial N}{\partial v} \Delta v + \frac{\partial N}{\partial p} \Delta p + \frac{\partial N}{\partial r} \Delta r + \frac{\partial N}{\partial \delta_r} \Delta \delta_r + \frac{\partial N}{\partial \delta_a} \Delta \delta_a
\end{aligned} \tag{3.2.1}$$

3.2.1 Extracting Lateral Stability Derivatives

In [A.8](#) we extracted the nondashed derivatives from the dashed ones.

3.2.2 Airframe Model

In [A.7](#) we construct the Controls derivatives and Stability derivatives matrices and get the change of the forces and moments then

calculating the total force acting on the airplane (this value is the input which given to the [A.4](#))

$$\begin{aligned}
\begin{Bmatrix} F_x \\ F_y \\ F_z \\ L \\ M \\ N \end{Bmatrix} &= \begin{Bmatrix} \Delta X \\ \Delta Y \\ \Delta Z \\ \Delta L \\ \Delta M \\ \Delta N \end{Bmatrix} + \begin{Bmatrix} X_0 \\ Y_0 \\ Z_0 \\ L_0 \\ M_0 \\ N_0 \end{Bmatrix} + \begin{Bmatrix} -mg \sin \theta \\ mg \cos \theta \sin \emptyset \\ mg \cos \theta \cos \emptyset \\ 0 \\ 0 \\ 0 \end{Bmatrix} \\
\begin{Bmatrix} X_0 \\ Y_0 \\ Z_0 \end{Bmatrix} &= \begin{Bmatrix} mg \sin \theta_0 \\ -mg \cos \theta_0 \sin \emptyset_0 \\ -mg \cos \theta_0 \cos \emptyset_0 \end{Bmatrix} \& \begin{Bmatrix} L_0 \\ M_0 \\ N_0 \end{Bmatrix} = \begin{Bmatrix} 0 \\ 0 \\ 0 \end{Bmatrix}
\end{aligned} \tag{3.2.2}$$

3.3 Results

Comparing the Simulink and RK4 code results yields that the results are identical validating the code results

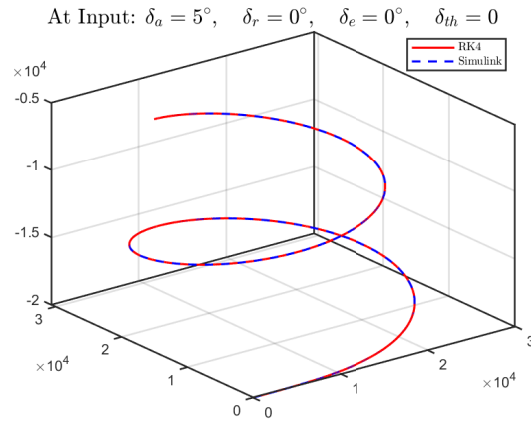
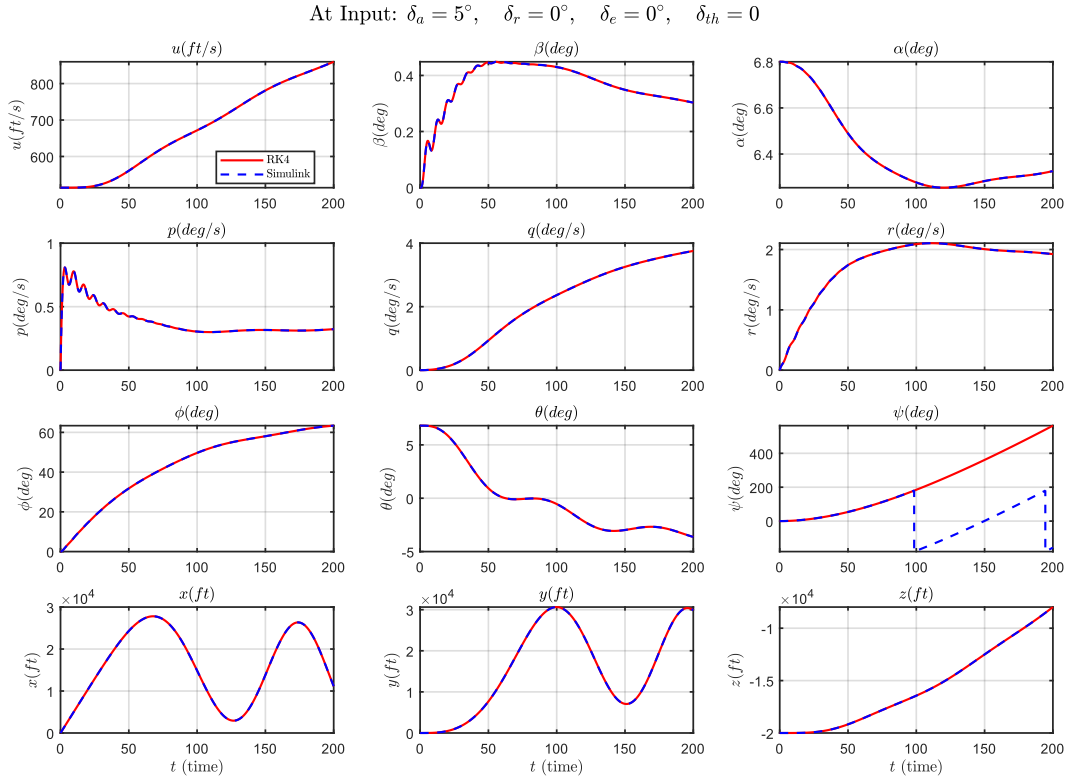


Figure 3.3.1: B747 Benchmark results at 5 degree aileron deflection

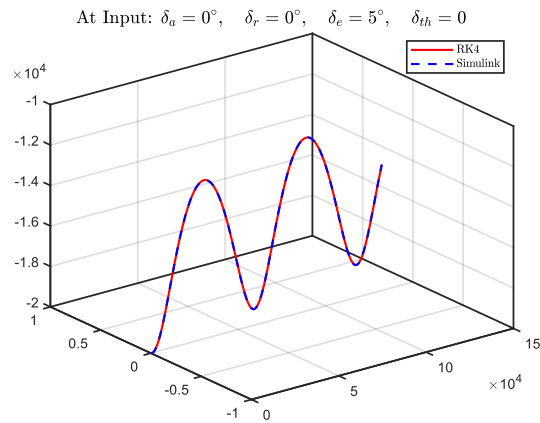
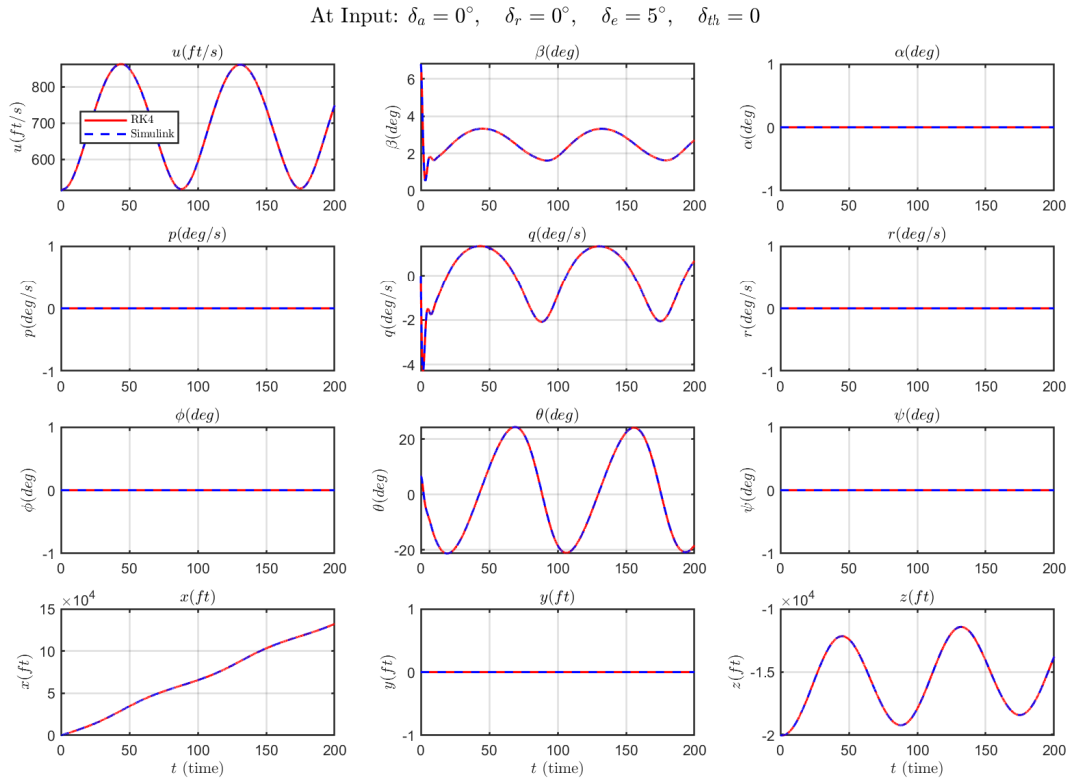
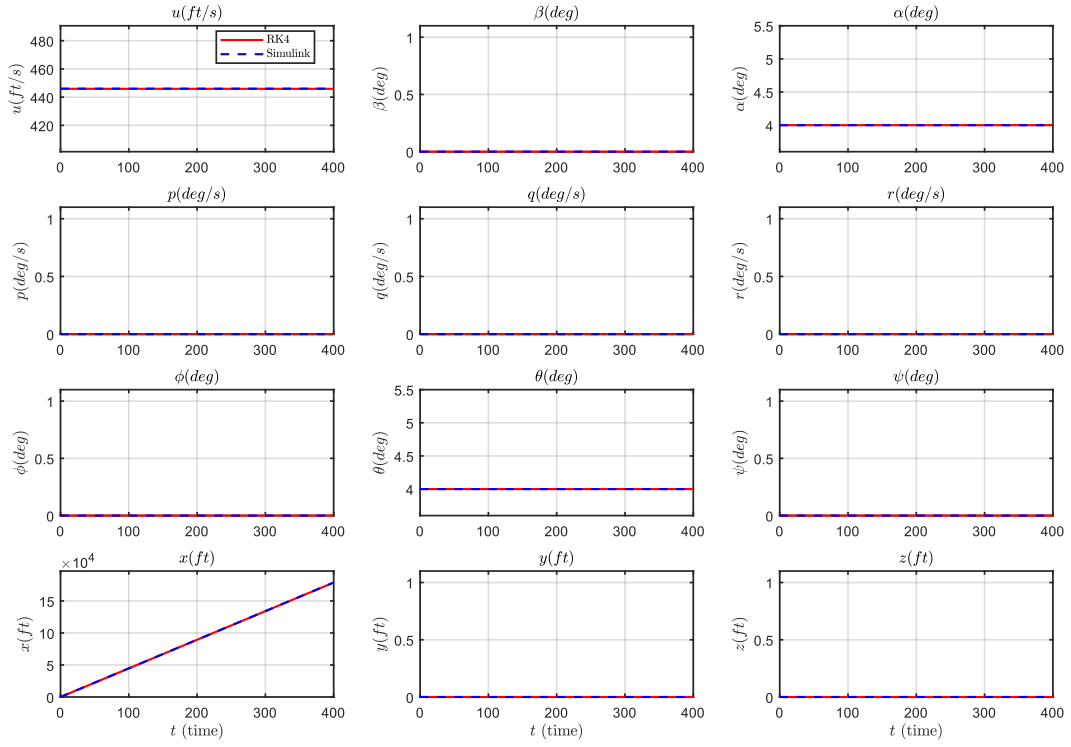


Figure 3.3.2: B747 Benchmark results at 5 degree elevator deflection

At Input: $\delta_a = 0^\circ$, $\delta_r = 0^\circ$, $\delta_e = 0^\circ$, $\delta_{th} = 0$



At Input: $\delta_a = 0^\circ$, $\delta_r = 0^\circ$, $\delta_e = 0^\circ$, $\delta_{th} = 0$

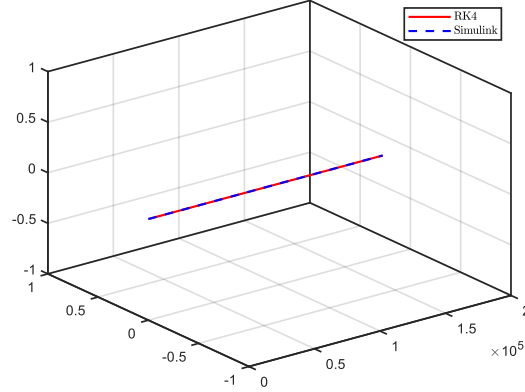
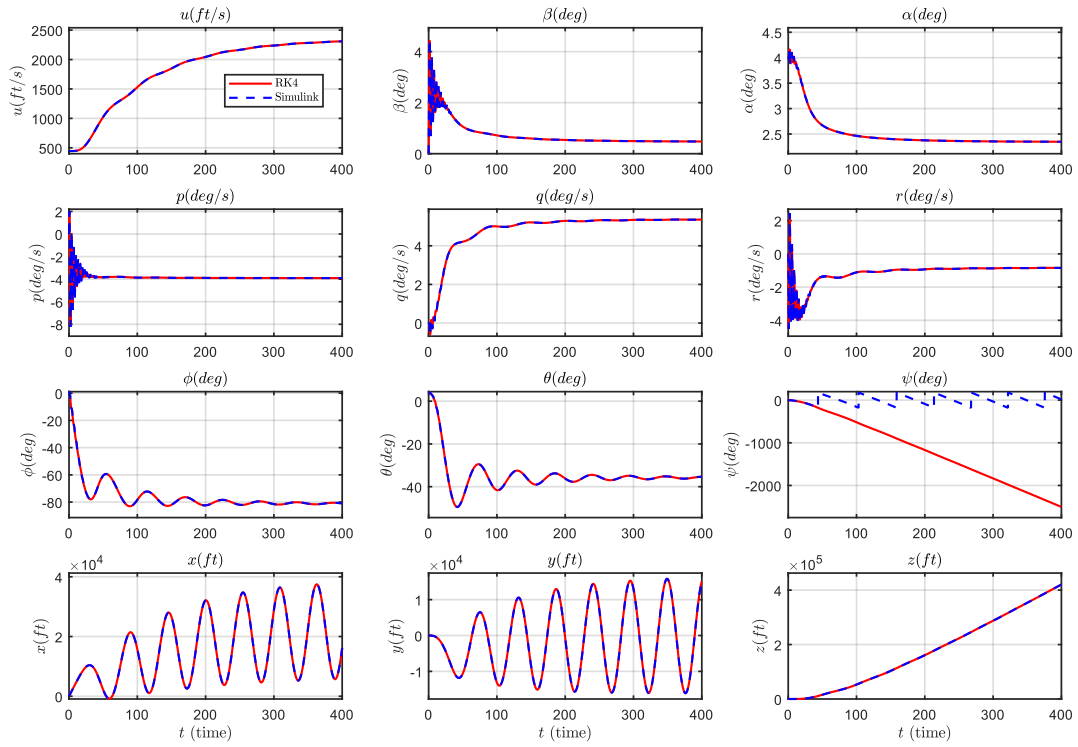


Figure 3.3.3: JetStar results at no input

At Input: $\delta_a = 0^\circ$, $\delta_r = 5^\circ$, $\delta_e = 0^\circ$, $\delta_{th} = 0$



At Input: $\delta_a = 0^\circ$, $\delta_r = 5^\circ$, $\delta_e = 0^\circ$, $\delta_{th} = 0$

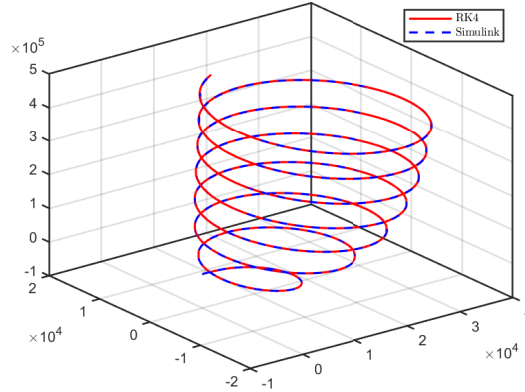
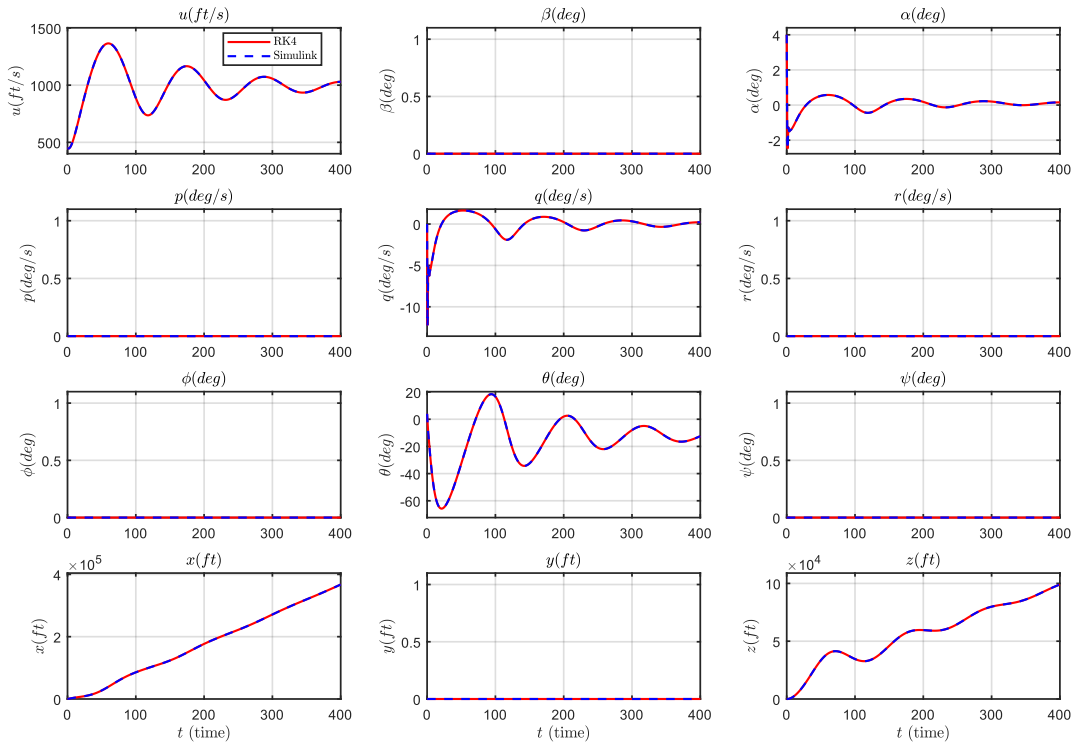


Figure 3.3.4: JetStar results at 5 degree rudder deflection

At Input: $\delta_a = 0^\circ$, $\delta_r = 0^\circ$, $\delta_e = 5^\circ$, $\delta_{th} = 0$



At Input: $\delta_a = 0^\circ$, $\delta_r = 0^\circ$, $\delta_e = 5^\circ$, $\delta_{th} = 0$

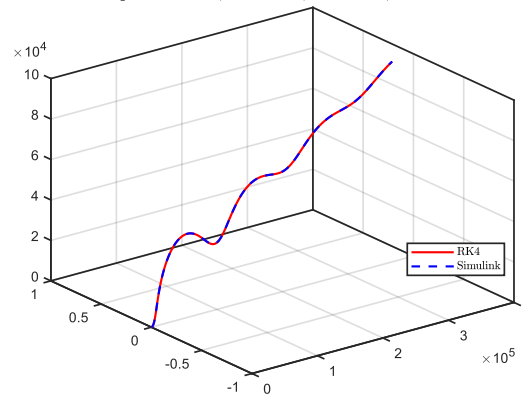
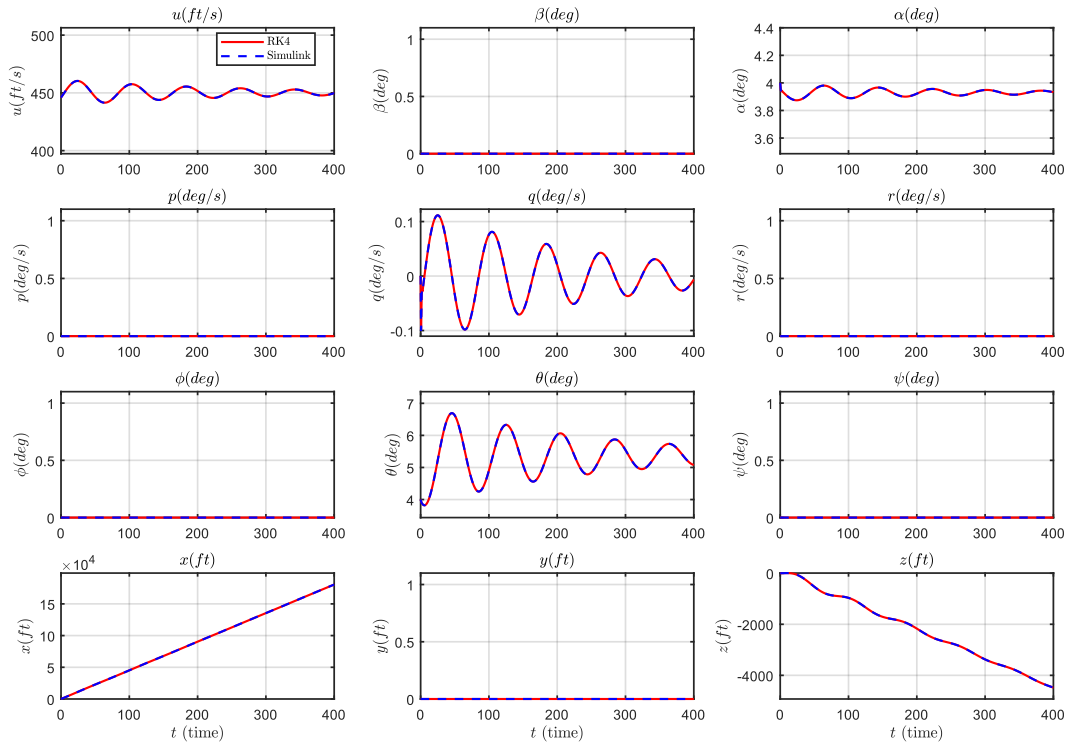


Figure 3.3.5: JetStar results at 5 degree elevator deflection

At Input: $\delta_a = 0^\circ$, $\delta_r = 0^\circ$, $\delta_e = 0^\circ$, $\delta_{th} = 1000$



At Input: $\delta_a = 0^\circ$, $\delta_r = 0^\circ$, $\delta_e = 0^\circ$, $\delta_{th} = 1000$

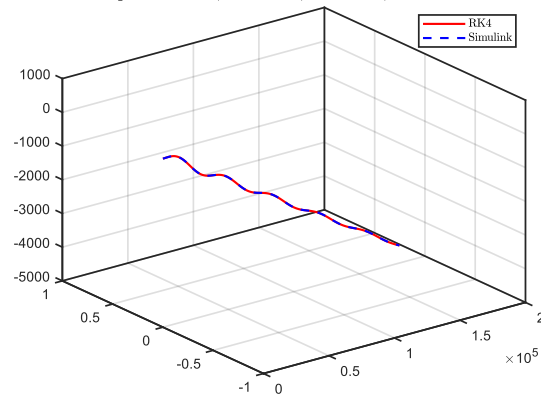


Figure 3.3.6: JetStar results at 1000lbf δ Thrust

Appendix A

MATLAB Codes

Listing A.1: problem1

```
1  clc
2  clearvars
3
4  % Initial conditions
5  t0 = 0;
6  Y0 = [2; 1];
7
8  % Solution parameters
9  tf = 20; % End time
10 n = 100; % Number of intervals
11 dt = (tf - t0) / n; % Step size
12 timespan = linspace(t0, tf, n+1);
13 Y_vec = zeros(2,n+1) ;
14 Y_vec(:,1) = Y0 ;
15
16 %Derivatives equation
17 f_vec = @(t,Y_vec) [sin(t) + cos(Y_vec(1)) + cos(Y_vec(2)) ; sin(t) + sin(Y_vec(2))];
18
19
20 %-----Solving the ODE using Runge-Kutta Method-----%
21
22 [t_vec_kutta,y_vec_kutta] = RK4(f_vec,timespan,Y0) ;
23
24 %-----Solving the ODE using Matlab ode45 Built in function-----%
25
26 [t_vec_ode_45,y_vec_ode_45] = ode45(f_vec,timespan,Y0) ;
27
28
29 figure;
30 subplot(1,2,1);
```

```

31 plot(t_vec_kutta, y_vec_kutta(1,:), 'r', 'LineWidth', 1.5); hold on;
32 plot(t_vec_ode_45, y_vec_ode_45(:,1), 'b', 'LineWidth', 1.5); hold on;
33 xlabel('t');
34 ylabel('y');
35 title('Solution of Y(1)');
36 legend('y1(t) using RK4 Method', 'y1(t) Using Matlab ode45 function');
37 grid on;
38 hold off;
39
40 subplot(1,2,2)
41 plot(t_vec_kutta, y_vec_kutta(2,:), 'r', 'LineWidth', 1.5); hold on;
42 plot(t_vec_ode_45, y_vec_ode_45(:,2), 'b', 'LineWidth', 1.5);
43 xlabel('t');
44 ylabel('y');
45 title('Solution of Y(2)');
46 legend('y2(t) using RK4 Method', 'y2(t) Using Matlab ode45 function');
47 grid on;
48 hold off;
49
50
51
52 ERROR_1 = mean(abs((y_vec_kutta(1,:).'-y_vec_ode_45(:,1))./y_vec_ode_45(:,1)))*100
53
54 ERROR_2 = mean(abs((y_vec_kutta(2,:).'-y_vec_ode_45(:,2))./y_vec_ode_45(:,2)))*100

```

Listing A.2: RK4

```

1 function [t_vec,y] = RK4(states_dot,t_vec,y0)
2
3 n = length(t_vec);
4 y(:,1) = y0 ;
5 dt = t_vec(2) -t_vec(1) ;
6
7 for i = 1:n
8
9     k1 = states_dot(t_vec(i),y(:,i)) ;
10    k2 = states_dot(t_vec(i)+dt/2,y(:,i)+k1*dt/2) ;
11    k3 = states_dot(t_vec(i)+dt/2,y(:,i)+k2*dt/2) ;
12    k4 = states_dot(t_vec(i)+dt,y(:,i)+k3*dt) ;
13
14    y(:,i+1) = y(:,i) + (dt/6)*(k1+2*k2+2*k3+k4) ;
15 end
16
17 y = y(:,1:n);

```

Listing A.3: AIRPLANE SIMULATOR PART I

```

1  clc;
2  clearvars;
3
4  %% ===== SYSTEM PARAMETERS =====
5  F = [2;8;3]; % External force vector (N)
6  M = [14;20;7]; % External moment vector (Nm)
7  I = [1, -2, -1; -2, 5, -4; -1, -4, 0.2]; % Inertia Matrix (kg*m^2)
8  m = 11; % Mass (kg)
9
10 %% ===== SIMULATION PARAMETERS =====
11 t0 = 0; % Initial time (s)
12 tf = 25; % Final time (s)
13 n = 20000; % Number of time steps
14 dt = (tf - t0) / (n - 1); % Time step size
15 timespan = t0:dt:tf; % Time vector
16
17 %% ===== INITIAL CONDITIONS =====
18 % State vector: [u, v, w, p, q, r, phi, theta, psi, x, y, z]
19 States_initial = [10; 2; 0; 2*pi/180; pi/180; 0; ...
20                  20*pi/180; 15*pi/180; 30*pi/180; 2; 4; 7];
21
22 % Extract individual initial states for clarity
23 u_0 = States_initial(1); v_0 = States_initial(2); w_0 = States_initial(3);
24 p_0 = States_initial(4); q_0 = States_initial(5); r_0 = States_initial(6);
25 phi_0 = States_initial(7); theta_0 = States_initial(8); epsi_0 = States_initial(9);
26 x_0 = States_initial(10); y_0 = States_initial(11); z_0 = States_initial(12);
27
28 %% ===== SOLVE USING RUNGE-KUTTA 4 =====
29 States_dot_fn = @get_states_dot;
30 [t_vec_kutta, states_vec_kutta] = RK4(States_dot_fn, timespan, States_initial);
31
32 %% ===== SOLVE USING SIMULINK =====
33 sim('sim_6_dof.slx');
34 y_vec_simulink = [uvw_out, anglerates_out, angles_out, xyz_out];
35
36 %% ===== PLOT RESULTS =====
37 plotStateComparisons(t_vec_kutta, states_vec_kutta, tout, y_vec_simulink);
38
39 %% ===== ERROR ANALYSIS =====
40 [abs_error, rmse, max_error] = compare_errors(t_vec_kutta, states_vec_kutta, tout',
        y_vec_simulink');

```

Listing A.4: getStatesDot

```

1 function states_dot = get_states_dot(t,states_vec,F,M,I,m)
2
3 u = states_vec(1);
4 v= states_vec(2);
5 w= states_vec(3);
6
7
8 p= states_vec(4);
9 q= states_vec(5);
10 r= states_vec(6);
11
12
13 phi= states_vec(7);
14 ceta= states_vec(8);
15 psi= states_vec(9);
16 eul = [psi ceta phi] ;
17
18 X= states_vec(10);
19 Y= states_vec(11);
20 Z= states_vec(12);
21
22
23 states_dot (1:3,1) = (1/m)*F-cross([p;q;r],[u;v;w]) ;
24
25 states_dot (4:6,1) = inv(I)*[M-cross([p;q;r],I*[p;q;r])] ;
26
27 states_dot (7:9,1) = [1, sin(phi)*tan(ceta), cos(phi)*tan(ceta) ; 0, cos(phi), -sin(phi)
    ; 0, sin(phi)/cos(ceta), cos(phi)/cos(ceta)] * [p;q;r];
28
29 states_dot (10:12,1) = eul2rotm(eul,"ZYX")*[u;v;w];
30
31
32
33
34
35 states_dot = states_dot (1:12,1);
36
37 end

```

Listing A.5: plotStateComparisons

```

1 function plotStateComparisons(t_vec_kutta, states_vec_kutta, t_vec_ode_45, y_vec_ode_45)
2     % Define state labels in lowercase (LaTeX format) and filenames

```

```

3   state_labels_latex = {'u', 'v', 'w', 'p', 'q', 'r', '\phi', '\theta', '\psi', 'x', 'y',
   , 'z'};
4   state_labels_filename = {'u', 'v', 'w', 'p', 'q', 'r', 'phi', 'theta', 'psi', 'x', 'y',
   , 'z'};
5
6   % Define colors and line styles
7   rk4_style = {'r-', 'LineWidth', 2}; % Red solid line for RK4
8   ode45_style = {'b--', 'LineWidth', 2}; % Blue dashed line for ode45
9
10  % Loop through all 12 states
11  for i = 1:12
12      figure;
13
14      % Plot RK4 and ode45 solutions
15      plot(t_vec_kutta, states_vec_kutta(i, :), rk4_style{:}); hold on;
16      plot(t_vec_ode_45, y_vec_ode_45(:, i), ode45_style{:});
17
18      % Set labels with LaTeX in lowercase
19      xlabel('$t_{time}$', 'Interpreter', 'latex', 'FontSize', 14);
20      ylabel(['$', state_labels_latex{i}, '$'], 'Interpreter', 'latex', 'FontSize', 14);
21      title(['solution_{of_}', state_labels_latex{i}, '(t)$'], 'Interpreter', 'latex', '
   FontSize', 16);
22
23      % Add legend
24      legend(['$', state_labels_latex{i}, '(t)_{using_}rk4'], ...
25             ['$', state_labels_latex{i}, '(t)_{using_}ode4(SIMULINK)'], ...
26             'Interpreter', 'latex', 'FontSize', 12, 'Location', 'Best');
27
28      % Beautify grid and axis
29      grid on;
30      set(gca, 'FontSize', 12, 'LineWidth', 1.2, 'Box', 'on');
31
32      % Save the figure as an SVG file with lowercase names
33      filename = sprintf('solution_%s.svg', state_labels_filename{i});
34      saveas(gcf, filename, 'svg');
35
36      hold off;
37  end
38 end

```

Listing A.6: compareErrors

```

1   function [abs_error, rmse, max_error] = compare_errors(t_rk4, y_rk4, t_ode45, y_ode45)
2
3   % Compute Absolute Error

```

```

4   abs_error = abs(y_rk4 - y_ode45);
5
6   % Compute Mean Absolute Error (MAE) for each state separately
7   mae = mean(abs_error, 2); % Mean across time steps (row-wise for each state)
8   % Compute RMSE (for each state separately)
9   rmse = sqrt(mean(abs_error.^2, 2)); % Row-wise RMSE (each row = one state)
10
11  % Compute Max Error (for each state separately)
12  max_error = max(abs_error, [], 2); % Max across time steps
13
14  % Display Results
15  disp('Mean_Absolute_Error_(MAE)_for_each_state:');
16  disp(mae);
17  disp('Root_Mean_Square_Error_(RMSE)_for_each_state:');
18  disp(rmse);
19  disp('Max_Error_for_each_state:');
20  disp(max_error);
21 end

```

Listing A.7: Airframe Model

```

1
2 function [F, M, stabilityDerivatives, controlsDerivatives] = afModel(SD_Long_final,
   SD_Lat_final, cuStates, states0, in, m, g, I)
3 % ACModel
4 Ixx = I(1);
5 Iyy = I(5);
6 Izz = I(9);
7
8 Xu = SD_Long_final(1);
9 Xw = SD_Long_final(4);
10 Xde = SD_Long_final(11);
11 Xdth = SD_Long_final(14);
12 Zu = SD_Long_final(2);
13 Zw = SD_Long_final(5);
14 Zwdot = SD_Long_final(7);
15 Zq = SD_Long_final(8);
16 Zde = SD_Long_final(12);
17 Zdth = SD_Long_final(15);
18 Mu = SD_Long_final(3);
19 Mw = SD_Long_final(6);
20 Mwdot = SD_Long_final(9);
21 Mq = SD_Long_final(10);
22 Mde = SD_Long_final(13);
23 Mdth = SD_Long_final(16);

```



```

24
25 % Lateral Stability Derivatives
26
27 Yv = SD_Lat_final(1);
28 Yb = SD_Lat_final(2);
29 Yp=0;
30 Yr=0;
31 Lv =SD_Lat_final(3);
32 Nv =SD_Lat_final(4);
33 Lp = SD_Lat_final(5);
34 Np = SD_Lat_final(6);
35 Lr = SD_Lat_final(7);
36 Nr = SD_Lat_final(8);
37 Yda = SD_Lat_final(9);
38 Ydr = SD_Lat_final(10);
39 Lda = SD_Lat_final(11);
40 Nda = SD_Lat_final(12);
41 Ldr = SD_Lat_final(13);
42 Ndr = SD_Lat_final(14);
43
44 %deltas
45
46 du = cuStates(1)-states0(1);
47 dv = cuStates(2)-states0(2);
48 dw = cuStates(3)-states0(3);
49 dp = cuStates(4)-states0(4);
50 dq = cuStates(5)-states0(5);
51 dr = cuStates(6)-states0(6);
52 dwdot = cuStates(13)-states0(13);
53
54 d_da = in(1);
55 d_dr = in(2);
56 d_de = in(3);
57 d_dth = in(4);
58
59
60
61 % Stability Derivatives matrix
62 dstates=[du dv dw dp dq dr dwdot]';
63 stabilityDerivatives=[ Xu 0 Xw 0 0 0 0
64                        0 Yv 0 Yp 0 Yr 0
65                        Zu 0 Zw 0 Zq 0 Zwdot
66                        0 Lv 0 Lp 0 Lr 0
67                        Mu 0 Mw 0 Mq 0 Mwdot
68                        0 Nv 0 Np 0 Nr 0 ];

```

```

69 % Control Matrix
70
71 dcontrols = [d_da d_de d_dth d_dr]';
72 % dcontrols = [d_da d_dr d_de d_dth]';
73
74 controlsDerivatives = [0 Xde Xdth 0
75                        Yda 0 0 Ydr
76                        0 Zde Zdth 0
77                        Lda 0 0 Ldr
78                        0 Mde Mdth 0
79                        Nda 0 0 Ndr];
80
81 delta=stabilityDerivatives*dstates + controlsDerivatives*dcontrols;
82
83 delta_F = m*delta(1:3);
84 delta_M = [Ixx;Iyy;Izz].*delta(4:6);
85
86 gravity0 = [m*g*sin(states0(8))
87             -m*g*cos(states0(8))*sin(states0(7))
88             -m*g*cos(states0(8))*cos(states0(7))];
89 gravity = [ -m*g*sin(cuStates(8))
90             m*g*cos(cuStates(8))*sin(cuStates(7))
91             m*g*cos(cuStates(8))*cos(cuStates(7))];
92 F = delta_F + gravity0 + gravity;
93 M = delta_M ;

```

Listing A.8: Lateral Function

```

1 % dash derivatives to normal derivatives Script
2
3 %% Step 1: Compute transformation matrix G and its inverse
4 G = 1 / (1 - (Ixz^2 / (Ixx * Izz)));
5 Trans_Mat = [G, G * Ixz / Ixx; G * Ixz / Izz, G];
6 Trans_Mat_inv = inv(Trans_Mat);
7
8 %% Step 2: Extract aerodynamic derivatives
9 Y_v = SD_Lat_dash(1); % Lateral velocity derivative
10 Y_beta = SD_Lat_dash(2); % Sideslip angle derivative
11 Y_da = SD_Lat_dash(9) * Vto; % Aileron control derivative
12 Y_dr = SD_Lat_dash(10) * Vto; % Rudder control derivative
13
14 %% Step 3: Convert L_N_dash into a single vector
15 L_N_dash_vector = [
16     SD_Lat_dash(3); SD_Lat_dash(4); % Beta derivatives
17     SD_Lat_dash(5); SD_Lat_dash(6); % Roll rate derivatives

```

```

18     SD_Lat_dash(7); SD_Lat_dash(8); % Yaw rate derivatives
19     SD_Lat_dash(11); SD_Lat_dash(12); % Aileron derivatives
20     SD_Lat_dash(13); SD_Lat_dash(14) % Rudder derivatives
21 ];
22
23 %% Step 4: Create a block diagonal transformation matrix
24 blk_diag_mat = blkdiag(Trans_Mat_inv, Trans_Mat_inv, Trans_Mat_inv, Trans_Mat_inv,
    Trans_Mat_inv);
25
26 %% Step 5: Compute LN_vector using the block diagonal matrix
27 LN_vector = blk_diag_mat * L_N_dash_vector;
28
29 %% Step 6: Construct the final SD_Lat vector
30 SD_Lat = [
31     Y_v;
32     Y_beta;
33     LN_vector(1:2)./Vto; % Assuming beta=v/Vto (small angle assumption)
34     LN_vector(3:6);
35     Y_da;
36     Y_dr;
37     LN_vector(7:end)
38 ];

```

Listing A.9: plotStateComparisons

```

1 function plotStateComparisons(t_vec_kutta, states_vec_kutta, t_vec_simulink,
    y_vec_simulink, dc)
2 % PLOTSTATECOMPARISONS Plots state comparisons between RK4 and Simulink results.
3 % This function generates a 3D trajectory plot and individual state comparison plots.
4 % It also saves the figures as SVG files.
5
6 %% Define state labels (LaTeX format and filenames)
7 state_labels_latex = {'u_{ft/s}', '\beta_{deg}', '\alpha_{deg}', 'p_{deg/s}', 'q_{deg/s}',
    , 'r_{deg/s}', ...
8     '\phi_{deg}', '\theta_{deg}', '\psi_{deg}', 'x_{ft}', 'y_{ft}', 'z_{ft}'};
9 state_labels_filename = {'u', 'beta', 'alpha', 'p', 'q', 'r', 'phi', 'theta', 'psi', 'x',
    'y', 'z'};
10 dc=dc*180/pi;
11 %% Define plot styles
12 rk4_style = {'r-', 'LineWidth', 1.5}; % Red solid line for RK4
13 simulink_style = {'b--', 'LineWidth', 1.5}; % Blue dashed line for Simulink
14
15 %% Plot 3D Trajectory Comparison
16 figure;

```

```

17 plot3(states_vec_kutta(10, :), states_vec_kutta(11, :), states_vec_kutta(12, :),
    rk4_style{:}); hold on;
18 plot3(y_vec_simulink(:, 10), y_vec_simulink(:, 11), y_vec_simulink(:, 12), simulink_style
    {:});
19 legend({'RK4', 'Simulink'}, 'Interpreter', 'latex', 'FontSize', 8, 'Location', 'Best');
20 grid on;
21 set(gca, 'FontSize', 10, 'LineWidth', 1.2, 'Box', 'on');
22
23 % Create a title string with control inputs
24 inputTitle = sprintf('\delta_a=\%.0f^\circ, \delta_r=\%.0f^\circ, \delta_e=\%.0f^\circ, \delta_{th}=\%.0f', ...
    dc(1), dc(2), dc(3), dc(4)*pi/180);
25 sgtitle(['\text{At Input: ', '$$', inputTitle, '$$'], 'Interpreter', 'latex');
26
27
28 % Save the trajectory plot as an SVG file
29 saveas(gcf, 'Trajectory.svg', 'svg');
30
31 %% Plot Individual State Comparisons
32 figure;
33 set(gcf, 'Position', [100, 100, 1200, 800]); % Adjust figure size
34
35 for i = 1:12
36     subplot(4, 3, i); % Arrange plots in a 4x3 grid
37
38     % Plot RK4 and Simulink data
39     plot(t_vec_kutta, states_vec_kutta(i, :), rk4_style{:}); hold on;
40     plot(t_vec_simulink, y_vec_simulink(:, i), simulink_style{:});
41
42     % Compute min and max values
43     y_max = max([max(states_vec_kutta(i, :)), max(y_vec_simulink(:, i))]);
44     y_min = min([min(states_vec_kutta(i, :)), min(y_vec_simulink(:, i))]);
45
46     % Ensure limits are increasing and valid
47     if y_max <= y_min
48         y_max = y_min + 1; % Add a small offset to avoid issues
49     end
50
51     % Set limits with margin
52     ylim([y_min - 0.1 * abs(y_min), y_max + 0.1 * abs(y_max)]);
53     % Set title and labels with LaTeX formatting
54     title(['$', state_labels_latex{i}, '$'], 'Interpreter', 'latex', 'FontSize', 10);
55     ylabel(['$', state_labels_latex{i}, '$'], 'Interpreter', 'latex', 'FontSize', 10);
56
57     % Show x-axis label only for bottom row
58     if i >= 10

```

```

59     xlabel('$t_{\square}(time)$', 'Interpreter', 'latex', 'FontSize', 10);
60 end
61
62 % Add legend only for the first subplot
63 if i == 1
64     legend({'RK4', 'Simulink'}, 'Interpreter', 'latex', 'FontSize', 8, 'Location', '
        Best');
65 end
66
67 grid on;
68 set(gca, 'FontSize', 10, 'LineWidth', 1.2, 'Box', 'on');
69 end
70
71 % Set the main title with LaTeX interpreter
72 sgtitle(['At_{\square}Input:_{\square}', '$$', inputTitle, '$$'], 'Interpreter', 'latex');
73
74 % Save the state comparisons as an SVG file
75 saveas(gcf, 'state_comparisons.svg', 'svg');
76
77 end

```

Listing A.10: AIRPLANE SIMULATOR PART II

```

1 % The aerodynamic forces and moments can be expressed as a function of all the motion
  variables [Nelson] page 63
2
3 %% Step 1: Clear workspace and close figures
4 clc;
5 clear all;
6 close all;
7
8 %% Step 2: Load Excel Sheet Data
9 filename_density_L = 'JetstarFC3'; %% Specify the location of your Excel sheet
10
11 aircraft_data = xlsread(filename_density_L, 'B2:B61'); %% Read data from Excel (B2:B61)
12
13 %% Step 3: Extract Time Vector Parameters
14 dt = aircraft_data(1); % Time step
15 tfinal = aircraft_data(2); % Final time
16 lengths = tfinal/dt + 1; % Total number of time steps
17 time_V = (0:dt:tfinal)'; % Time vector
18
19 %% Step 4: Extract Initial Conditions
20 s0 = aircraft_data(4:15); % Initial state variables
21 wdot0 = 0; % Initial angular acceleration

```

```

22 | sdot0 = zeros(12,1); % Zero initial state derivatives
23 | Vto = sqrt(s0(1)^2 + s0(2)^2 + s0(3)^2); % Initial velocity magnitude
24 | initial_conditions = [aircraft_data(13:15)' aircraft_data(10:12)' aircraft_data(4:6)'
    |   aircraft_data(7:9)' aircraft_data(18) aircraft_data(20) Vto 0] ;
25 | %% Step 5: Define Control Inputs
26 | dc = [aircraft_data(57:59)* pi/180; aircraft_data(60)]; % Control surface deflections
27 | da= dc(1);
28 | dr= dc(2);
29 | de= dc(3);
30 | dth=dc(4);
31 | %% Step 6: Define Gravity, Mass, and Inertia Parameters
32 | m = aircraft_data(51); % Mass
33 | g = aircraft_data(52); % Gravity acceleration
34 | Ixx = aircraft_data(53);
35 | Iyy = aircraft_data(54);
36 | Izz = aircraft_data(55);
37 | Ixz = aircraft_data(56);
38 | Ixy = 0;
39 | Iyz = 0;
40 | I = [Ixx, -Ixy, -Ixz;
    |   -Ixy, Iyy, -Iyz;
    |   -Ixz, -Iyz, Izz]; % Inertia matrix
43 | invI = inv(I); % Inverse of inertia matrix
44 | % initial gravity force
45 | F_gravity_0 = m*g * [ sin(s0(8)) ;
    |                   -cos(s0(8))*sin(s0(7)) ;
    |                   -cos(s0(8))*cos(s0(7)) ];
48 | M0 = [0; 0; 0];
49 | %% Step 7: Load Stability Derivatives
50 | % Longitudinal Motion Derivatives
51 | SD_Long = aircraft_data(21:36);
52 | SD_Long_final = SD_Long;
53 |
54 | % Lateral Motion Derivatives
55 | SD_Lat_dash = aircraft_data(37:50);
56 | run('LateralFunc'); % Run lateral derivatives transformation script
57 | SD_Lat_final = SD_Lat;
58 |
59 | %% Step 8: Solve Equations of Motion using RK4 Method
60 | states(:,1) = s0;
61 | for i = 1:lengths
62 |     if i == 1
63 |         wdot = wdot0;
64 |     end
65 |

```

```

66     % Compute Forces and Moments
67     [F, M, Matrix_states, Matrix_Controls] = afModel(SD_Long_final, SD_Lat_final, [states(:,i)
        ]; wdot], [s0; wdot0], dc, m, g, I);
68
69     % Runge-Kutta 4th Order Method (RK4)
70     K_1 = get_states_dot(time_V(i), states(:,i), F, M, I, m);
71     K_2 = get_states_dot(time_V(i) + dt/2, states(:,i) + dt*K_1/2, F, M, I, m);
72     K_3 = get_states_dot(time_V(i) + dt/2, states(:,i) + dt*K_2/2, F, M, I, m);
73     K_4 = get_states_dot(time_V(i) + dt, states(:,i) + dt*K_3, F, M, I, m);
74
75     % Update State Variables
76     states(:,i+1) = states(:,i) + (dt/6) * (K_1 + 2*K_2 + 2*K_3 + K_4);
77     wdot = K_1(3);
78 end
79
80 states = states(:,1:lengths);
81
82 %% Step 10: States Rearrangement
83
84 u = states(1,:);
85 v = states(2,:);
86 w = states(3,:);
87 p_deg = states(4,:)*180/pi;
88 q_deg = states(5,:)*180/pi;
89 r_deg = states(6,:)*180/pi;
90 phi_deg = states(7,:)*180/pi;
91 theta_deg = states(8,:)*180/pi;
92 psi_deg = states(9,:)*180/pi;
93 x = states(10,:);
94 y = states(11,:);
95 z = round(states(12,:),5);
96 alpha_deg = atan(w./u)*180/pi;
97 beta_deg = asin(v./sqrt(u.^2+v.^2+w.^2))*180/pi;
98 states_vec_kutta = [
99     u
100    beta_deg;
101    alpha_deg;
102    p_deg;
103    q_deg;
104    r_deg;
105    phi_deg;
106    theta_deg;
107    psi_deg;
108    x;
109    y;

```

```
110     z;  
111  
112     ];  
113     %% simulation  
114     sim('Aircraft_Simulator.slx');  
115  
116     sim_vec = [  
117         out_u.Data';out_beta.Data'; out_alpha.Data'; out_p.Data'; out_q.Data'; out_r.Data';  
            out_phi.Data'; out_theta.Data'; out_psi.Data'; out_x.Data'; out_y.Data'; out_z.  
            Data'  
118     ];  
119     plotStateComparisons(time_V, states_vec_kutta, tout', sim_vec',dc)
```


Bibliography

- [1] J. Blakelock, *Automatic Control of Aircraft and Missiles*, ser. A Wiley-Interscience publication. Wiley, 1991. [Online]. Available: <https://books.google.com.eg/books?id=ubcczZUDCsMC>
- [2] B. Stevens, F. Lewis, and E. Johnson, *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. Wiley, 2015. [Online]. Available: <https://books.google.com.eg/books?id=boybCgAAQBAJ>
- [3] M. Sadraey, *Unmanned Aircraft Design Techniques*, ser. Aerospace Series. John Wiley & Sons Canada, Limited, 2020. [Online]. Available: <https://books.google.com.eg/books?id=IX9IzQEACAAJ>
- [4] G. F. Franklin, J. D. Powell, and M. L. Workman, *Digital Control of Dynamic Systems*, 3rd ed. Addison-Wesley, 2014.
- [5] K. Ogata, *Discrete-Time Control Systems*, 2nd ed. Pearson, 2010.
- [6] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Wiley, 2016.
- [7] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed. McGraw-Hill, 2015.
- [8] B. Etkin and L. D. Reid, *Dynamics of Flight: Stability and Control*, 3rd ed. Wiley, 1996.